
Prof. Ryan Cotterell, Prof. Mrinmaya Sachan, Prof. Florian Tramèr

Assignment 1

Goerkem Kadir Solun, 25-936-154

Collaborators

1. I prepared this document using VsCode, the LaTeX Workshop extension and Latexmk.
2. I got formatting help from VsCode's copilot.

Declaration

I, Goerkem Kadir Solun, confirm that I have written this assignment independently, without any direct copying or paraphrasing from other students' work. I have acknowledged everyone I consulted during my research. I understand that while consultation and comparison of solutions are permitted, the final written submission must be my own work.

I am aware that any form of plagiarism is a violation of academic integrity. If it is determined that I have engaged in academic dishonesty, I accept and understand that the university's standard measures for plagiarism will be applied.

By submitting this assignment, I affirm my commitment to academic honesty and integrity.

Question 1: Representation Surgery

a) Linear Guarding vs. Arbitrary Classifiers

No. By Definition 2, the statement that $\mathbf{X}$ *linearly guards* Z only guarantees that the trivially attainable loss L_τ is optimal over the set of all affine transformations and linear predictors. Mathematically, it ensures that $\mathbb{E}[\mathcal{L}(f(\mathbf{X}), Z)] \geq L_\tau$ holds for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$. However, the classifier $c : \mathbf{x} \mapsto c(\mathbf{x}) \in \mathcal{Z}$ proposed in the question is an *arbitrary* classifier, meaning it is not restricted to being linear. Because linear guarding only constrains linear probes, a non-linear classifier such as a multi-layer perceptron could draw non-linear decision boundaries to extract the concept information Z still embedded within $\mathbf{X}$. Consequently, such a non-linear classifier could successfully achieve a loss lower than the trivially attainable loss L_τ .

b) Class-Conditional Mean Equality Implies Linear Guarding

If each class-conditional mean satisfies $\mathbb{E}[\mathbf{X} \mid Z = i] = \mathbb{E}[\mathbf{X}]$, then $\mathbf{X}$ linearly guards Z w.r.t. the cross-entropy loss $\mathcal{L}$. To prove that $\mathbf{X}$ linearly guards Z , we must show that for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ (where $\mathbf{A} \in \mathbb{R}^{k \times d}$ and $\mathbf{b} \in \mathbb{R}^k$), the expected loss is bounded by the trivially attainable loss L_τ :

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \geq L_\tau \quad (1)$$

Rewrite the expected loss by conditioning on the class Z :

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] = \mathbb{E}_Z \left[\mathbb{E}_{\mathbf{X}|Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z) \mid Z] \right] \quad (2)$$

The cross-entropy loss $\mathcal{L}(\mathbf{y}, Z = i) = -\ln \frac{\exp(y_i)}{\sum_{j=1}^k \exp(y_j)}$ is convex w.r.t. its input logits $\mathbf{y}$. Because $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$ is an affine, the composition $\mathcal{L}(\mathbf{A}\mathbf{x} + \mathbf{b}, Z = i)$ remains convex w.r.t. $\mathbf{x}$. Applying Jensen's inequality gives:

$$\mathbb{E}_{\mathbf{X}|Z=i} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, i)] \geq \mathcal{L}(\mathbb{E}_{\mathbf{X}|Z=i} [\mathbf{A}\mathbf{X} + \mathbf{b}], i) = \mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X} \mid Z = i] + \mathbf{b}, i) \quad (3)$$

Use the fact that class-conditional means equal the global mean, $\mathbb{E}[\mathbf{X} \mid Z = i] = \mathbb{E}[\mathbf{X}]$.

$$\mathbb{E}_{\mathbf{X}|Z=i} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, i)] \geq \mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}, i) \quad (4)$$

Substituting this lower bound back into the outer expectation over Z gives:

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \geq \mathbb{E}_Z [\mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}, Z)] \quad (5)$$

Notice that $\mathbf{b}' \triangleq \mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}$ is simply a constant vector in $\mathbb{R}^k$, independent of $\mathbf{X}$. Thus, the right side of the inequality evaluates the loss of a *constant* predictor $f(\mathbf{x}) = \mathbf{b}'$.

The trivially attainable loss L_τ is the infimum of the expected loss over all constant predictors:

$$L_\tau = \inf_{\mathbf{c} \in \mathbb{R}^k} \mathbb{E}_Z [\mathcal{L}(\mathbf{c}, Z)] \quad (6)$$

Since $\mathbf{b}'$ is a constant in $\mathbb{R}^k$, its expected loss must be greater than or equal to the infimum:

$$\mathbb{E}_Z [\mathcal{L}(\mathbf{b}', Z)] \geq L_\tau \quad (7)$$

Combining the inequalities, we have shown that for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$:

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(f(\mathbf{X}), Z)] \geq L_\tau \quad (8)$$

Therefore, by Definition 2, $\mathbf{X}$ linearly guards Z w.r.t. the cross-entropy loss $\mathcal{L}$.

c) **Boundedness and Symmetry of the Cross-Entropy Gradient**

Let $\mathbf{x}$ be an observed representation, and $\mathbf{y} \triangleq \mathbf{Ax} + \mathbf{b}$.

All partial derivatives of the cross-entropy loss w.r.t. $\mathbf{y}$ are bounded. Recall the definition of the cross-entropy loss for logit vector $\mathbf{y}$ and true concept class $Z = c$:

$$\mathcal{L}(\mathbf{y}, Z = c) = -\ln \left(\frac{\exp(y_c)}{\sum_{j=1}^k \exp(y_j)} \right) = -y_c + \ln \left(\sum_{j=1}^k \exp(y_j) \right) \quad (9)$$

Compute the partial derivative of the loss w.r.t. an arbitrary logit y_m . **Case 1** ($m = c$):

$$\frac{\partial}{\partial y_c} \mathcal{L}(\mathbf{y}, Z = c) = -1 + \frac{\frac{\partial}{\partial y_c} \sum_{j=1}^k \exp(y_j)}{\sum_{j=1}^k \exp(y_j)} \quad (10)$$

$$= -1 + \frac{\exp(y_c)}{\sum_{j=1}^k \exp(y_j)} = -1 + \text{softmax}(\mathbf{y})_c \quad (11)$$

Since the softmax output lies in $(0, 1)$ for any $\mathbf{y}$, we have $-1 < \frac{\partial}{\partial y_c} \mathcal{L}(\mathbf{y}, Z = c) < 0$.

Case 2 ($m \neq c$):

$$\frac{\partial}{\partial y_m} \mathcal{L}(\mathbf{y}, Z = c) = 0 + \frac{\frac{\partial}{\partial y_m} \sum_{j=1}^k \exp(y_j)}{\sum_{j=1}^k \exp(y_j)} \quad (12)$$

$$= \frac{\exp(y_m)}{\sum_{j=1}^k \exp(y_j)} = \text{softmax}(\mathbf{y})_m \quad (13)$$

We have $0 < \frac{\partial}{\partial y_m} \mathcal{L}(\mathbf{y}, Z = c) < 1$. Because partial derivatives fall within the interval $(-1, 1)$, they are uniformly bounded.

For any $j_1, j_2 \neq i$, we have $\frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_1) = \frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_2) \neq 0$. Find the partial derivative of the loss w.r.t. y_i when the true class is $Z = j$ with $j \neq i$. Using the derivation from above, the partial derivatives are the i -th component of the softmax output:

$$\frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_1) = \frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_2) = \frac{\exp(y_i)}{\sum_{l=1}^k \exp(y_l)} \quad (14)$$

Both expressions are equal. Furthermore, because the exponential function e^x is positive for any real x , we have $\exp(y_i) > 0$. The denominator is a sum of positive exponentials, so it is greater than zero. Therefore:

$$\frac{\exp(y_i)}{\sum_{l=1}^k \exp(y_l)} > 0 \quad (15)$$

Thus, we have shown that:

$$\frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_1) = \frac{\partial}{\partial y_i} \mathcal{L}(\mathbf{y}, Z = j_2) \neq 0 \quad (16)$$

d) Linear Guarding Implies Equal Class-Conditional Means

Because $\mathbf{X}$ linearly guards Z , the expected loss for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ is bounded by the trivially attainable loss L_τ achieved by the constant predictor $\mathbf{b}^*$. Thus,

$$J(\mathbf{A}, \mathbf{b}) \triangleq \mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \quad (17)$$

attains its minimum at $\mathbf{A} = \mathbf{0}$ and $\mathbf{b} = \mathbf{b}^*$. By the first-order optimality, the gradient of J w.r.t. $\mathbf{A}$, evaluated at $\mathbf{A} = \mathbf{0}, \mathbf{b} = \mathbf{b}^*$:

$$\nabla_{\mathbf{A}} \mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \Big|_{\mathbf{A}=\mathbf{0}, \mathbf{b}=\mathbf{b}^*} = \mathbf{0} \quad (18)$$

Move the derivative inside the expectation to evaluate the gradient. Let $\mathbf{y} = \mathbf{A}\mathbf{X} + \mathbf{b}$.

$$\frac{\partial \mathcal{L}}{\partial A_{ij}} = \frac{\partial \mathcal{L}}{\partial y_i} \frac{\partial y_i}{\partial A_{ij}} = \frac{\partial \mathcal{L}}{\partial y_i} X_j. \quad (19)$$

In part c we proved that the partial derivatives of the cross-entropy loss w.r.t. $\mathbf{y}$ are bounded between -1 and 1 . Because the gradient w.r.t $\mathbf{y}$ is uniformly bounded, the Dominated Convergence Theorem allows us to interchange expectation and differentiation:

$$\nabla_{\mathbf{A}} J(\mathbf{A}, \mathbf{b}) = \mathbb{E}_{\mathbf{X}, Z} [\nabla_{\mathbf{A}} \mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] = \mathbb{E}_{\mathbf{X}, Z} [\nabla_{\mathbf{y}} \mathcal{L}(\mathbf{y}, Z) \mathbf{X}^\top] \quad (20)$$

From part c, for any logit y_m the partial derivative of the loss is $\frac{\partial \mathcal{L}}{\partial y_m} = \text{softmax}(\mathbf{y})_m - \mathbb{1}(Z = m)$. Let $\mathbf{p}(\mathbf{y})$ be the vector of softmax probabilities and let $\tilde{\mathbf{Z}}$ be the one-hot encoding of Z .

$$\nabla_{\mathbf{y}} \mathcal{L}(\mathbf{y}, Z) = \mathbf{p}(\mathbf{y}) - \tilde{\mathbf{Z}}. \quad (21)$$

Evaluating the gradient w.r.t. $\mathbf{A}$ at the optimum $\mathbf{A} = \mathbf{0}, \mathbf{b} = \mathbf{b}^*$ so $\mathbf{y} = \mathbf{b}^*$ yields:

$$\mathbb{E}_{\mathbf{X}, Z} [(\mathbf{p}(\mathbf{b}^*) - \tilde{\mathbf{Z}}) \mathbf{X}^\top] = \mathbf{0} \quad (22)$$

Restricting to a single class i , this implies

$$\mathbb{E}_{\mathbf{X}, Z} [(p_i(\mathbf{b}^*) - \mathbb{1}(Z = i)) \mathbf{X}] = \mathbf{0}. \quad (23)$$

By linearity of expectation,

$$p_i(\mathbf{b}^*) \mathbb{E}[\mathbf{X}] = \mathbb{E}[\mathbf{X} \mathbb{1}(Z = i)] \quad (24)$$

To determine $p_i(\mathbf{b}^*)$, recall that $\mathbf{b}^*$ minimizes the trivially attainable loss $L_\tau = \mathbb{E}_Z [\mathcal{L}(\mathbf{b}, Z)]$. First-order optimality w.r.t. $\mathbf{b}$ gives:

$$\nabla_{\mathbf{b}} \mathbb{E}_Z [\mathcal{L}(\mathbf{b}, Z)] = \mathbb{E}_Z [\nabla_{\mathbf{b}} \mathcal{L}(\mathbf{b}, Z)] = \mathbb{E}_Z [\mathbf{p}(\mathbf{b}) - \tilde{\mathbf{Z}}] = \mathbf{0} \quad (25)$$

Hence, at the optimum $\mathbf{b}^*$, the softmax probabilities must match the marginal class probabilities:

$$\mathbf{p}(\mathbf{b}^*) = \mathbb{E}_Z [\tilde{\mathbf{Z}}], \quad \text{and thus} \quad p_i(\mathbf{b}^*) = \mathbb{P}(Z = i). \quad (26)$$

Substituting $p_i(\mathbf{b}^*) = \mathbb{P}(Z = i)$ back into (24):

$$\mathbb{P}(Z = i) \mathbb{E}[\mathbf{X}] = \mathbb{E}[\mathbf{X} \mathbb{1}(Z = i)] \quad (27)$$

By the definition of conditional expectation, $\mathbb{E}[\mathbf{X} \mathbb{1}(Z = i)] = \mathbb{E}[\mathbf{X} | Z = i] \mathbb{P}(Z = i)$, so

$$\mathbb{P}(Z = i) \mathbb{E}[\mathbf{X}] = \mathbb{E}[\mathbf{X} | Z = i] \mathbb{P}(Z = i). \quad (28)$$

Since $\mathbb{P}(Z = i) \neq 0$ by assumption, we can divide both sides by $\mathbb{P}(Z = i)$ to obtain

$$\mathbb{E}[\mathbf{X}] = \mathbb{E}[\mathbf{X} | Z = i]. \quad (29)$$

This holds for every class i .

e) **Zero Cross-Covariance Characterizes Equal Class-Conditional Means**

The cross-covariance between $\mathbf{X}$ and $\tilde{\mathbf{Z}}$ is

$$\Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbb{E}[\mathbf{X} \tilde{\mathbf{Z}}^\top] - \mathbb{E}[\mathbf{X}] \mathbb{E}[\tilde{\mathbf{Z}}]^\top. \quad (30)$$

Its j^{th} column is the covariance of the vector $\mathbf{X}$ with the scalar $\tilde{Z}_j$:

$$\text{Cov}(\mathbf{X}, \tilde{Z}_j) = \mathbb{E}[\mathbf{X} \tilde{Z}_j] - \mathbb{E}[\mathbf{X}] \mathbb{E}[\tilde{Z}_j]. \quad (31)$$

Since $\tilde{Z}_j = \mathbb{1}(Z = j)$, we have $\mathbb{E}[\tilde{Z}_j] = \mathbb{P}(Z = j)$ and, by the definition of conditional expectation, $\mathbb{E}[\mathbf{X} \tilde{Z}_j] = \mathbb{E}[\mathbf{X} | Z = j] \mathbb{P}(Z = j)$. Substituting gives:

$$\text{Cov}(\mathbf{X}, \tilde{Z}_j) = \left(\mathbb{E}[\mathbf{X} | Z = j] - \mathbb{E}[\mathbf{X}] \right) \mathbb{P}(Z = j) \quad (32)$$

Forward direction Assume $\mathbb{E}[\mathbf{X} | Z = j] = \mathbb{E}[\mathbf{X}]$ for every j . Then the parenthesised term vanishes, so $\text{Cov}(\mathbf{X}, \tilde{Z}_j) = \mathbf{0}$ for every column j , and therefore $\Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}$.

Backward direction Assume $\Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}$. Every column must then be zero, so for each j :

$$\left(\mathbb{E}[\mathbf{X} | Z = j] - \mathbb{E}[\mathbf{X}] \right) \mathbb{P}(Z = j) = \mathbf{0}. \quad (33)$$

Since $\mathbb{P}(Z = j) \neq 0$ by assumption, the directions give $\mathbb{E}[\mathbf{X} | Z = j] = \mathbb{E}[\mathbf{X}] \iff \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}$.

f) **Linear Guarding under an Affine Map**

A representation linearly guards Z if and only if its cross-covariance with the one-hot encoded concept vector $\tilde{\mathbf{Z}}$ is zero. Applied to the transformed representation $r(\mathbf{X}) = \mathbf{P}\mathbf{X} + \mathbf{b}$:

$$r(\mathbf{X}) \text{ linearly guards } Z \iff \Sigma_{r(\mathbf{X}), \tilde{\mathbf{Z}}} = \mathbf{0}. \quad (34)$$

Adding the constant vector $\mathbf{b}$ only shifts the mean and does not change covariance, so

$$\text{Cov}(\mathbf{P}\mathbf{X} + \mathbf{b}, \tilde{\mathbf{Z}}) = \text{Cov}(\mathbf{P}\mathbf{X}, \tilde{\mathbf{Z}}). \quad (35)$$

For any constant matrix $\mathbf{P}$ and random vectors $\mathbf{X}, \mathbf{Y}$, $\text{Cov}(\mathbf{P}\mathbf{X}, \mathbf{Y}) = \mathbf{P} \text{Cov}(\mathbf{X}, \mathbf{Y})$.

$$\text{Cov}(\mathbf{P}\mathbf{X}, \tilde{\mathbf{Z}}) = \mathbf{P} \text{Cov}(\mathbf{X}, \tilde{\mathbf{Z}}) = \mathbf{P} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}. \quad (36)$$

Hence $\Sigma_{r(\mathbf{X}), \tilde{\mathbf{Z}}} = \mathbf{P} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}$. Combining with the guarding characterisation above yields:

$$r(\mathbf{X}) \text{ linearly guards } Z \iff \mathbf{P} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}. \quad (37)$$

g) **Decoupling the Distortion under a Centered Representation**

Define the distortion induced by the affine map $r(\mathbf{X}) = \mathbf{P}\mathbf{X} + \mathbf{b}$ as $D(\mathbf{P}, \mathbf{b}) \triangleq \mathbb{E}[\|\mathbf{P}\mathbf{X} + \mathbf{b} - \mathbf{X}\|^2]$.

Part 1: If $\mathbb{E}[\mathbf{X}] = \mathbf{0}$, then $D(\mathbf{P}, \mathbf{b}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2] + \|\mathbf{b}\|^2$. Group the terms with $\mathbf{X}$:

$$D(\mathbf{P}, \mathbf{b}) = \mathbb{E}[\|(\mathbf{P}\mathbf{X} - \mathbf{X}) + \mathbf{b}\|^2]. \quad (38)$$

Use $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + 2\langle \mathbf{u}, \mathbf{v} \rangle$ with $\mathbf{u} = \mathbf{P}\mathbf{X} - \mathbf{X}$ and $\mathbf{v} = \mathbf{b}$, and expectation linearity:

$$D(\mathbf{P}, \mathbf{b}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2] + \|\mathbf{b}\|^2 + 2\mathbb{E}[\langle \mathbf{P}\mathbf{X} - \mathbf{X}, \mathbf{b} \rangle]. \quad (39)$$

Since $\mathbf{b}$ is deterministic, $\mathbb{E}[\|\mathbf{b}\|^2] = \|\mathbf{b}\|^2$. Linearity of the inner product and of expectation gives:

$$\mathbb{E}[\langle \mathbf{P}\mathbf{X} - \mathbf{X}, \mathbf{b} \rangle] = \langle \mathbb{E}[\mathbf{P}\mathbf{X} - \mathbf{X}], \mathbf{b} \rangle = \langle (\mathbf{P} - \mathbf{I}) \mathbb{E}[\mathbf{X}], \mathbf{b} \rangle. \quad (40)$$

Since $\mathbb{E}[\mathbf{X}] = \mathbf{0}$, this reduces to $\langle \mathbf{0}, \mathbf{b} \rangle = 0$, and the cross term vanishes, leaving:

$$D(\mathbf{P}, \mathbf{b}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2] + \|\mathbf{b}\|^2. \quad (41)$$

Part 2: Minimizing $D(\mathbf{P}, \mathbf{b})$ subject to a constraint on $\mathbf{P}$ is equivalent to minimizing $D(\mathbf{P}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2]$. From part 1, when $\mathbb{E}[\mathbf{X}] = \mathbf{0}$ the distortion decouples into two non-negative terms:

$$D(\mathbf{P}, \mathbf{b}) = \underbrace{\mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2]}_{\text{depends only on } \mathbf{P}} + \underbrace{\|\mathbf{b}\|^2}_{\text{depends only on } \mathbf{b}}. \quad (42)$$

The constraint is only on $\mathbf{P}$, so $\mathbf{b}$ can be chosen freely. Since $\|\mathbf{b}\|^2 \geq 0$, the unconstrained optimum is $\mathbf{b}^* = \mathbf{0}$. Substituting back, the minimization reduces to

$$D(\mathbf{P}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2]. \quad (43)$$

h) Optimal Guarding Projection under Whitened Representations

Assume $\mathbb{E}[\mathbf{X}] = \mathbf{0}$ and $\Sigma_{\mathbf{X}, \mathbf{X}} = \mathbf{I}$. Show the following satisfies Eq. (7) and minimizes Eq. (8).

$$\mathbf{P}^* = \mathbf{I} - \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}^+ \quad (44)$$

Part 1: $\mathbf{P}^*$ satisfies the guarding constraint. Substitute $\mathbf{P}^*$ into the constraint:

$$\mathbf{P}^* \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} - \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}^+ \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}. \quad (45)$$

By the property of the pseudoinverse, $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$ for any matrix $\mathbf{A}$, so

$$\mathbf{P}^* \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} - \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}. \quad (46)$$

Part 2: $\mathbf{P}^*$ minimizes the distortion. From part (g), we minimize $D(\mathbf{P}) = \mathbb{E}[\|\mathbf{P}\mathbf{X} - \mathbf{X}\|^2] = \mathbb{E}[\|(\mathbf{P} - \mathbf{I})\mathbf{X}\|^2]$ subject to $\mathbf{P} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}$. Applying Hint 1 together with $\Sigma_{\mathbf{X}, \mathbf{X}} = \mathbf{I}$:

$$D(\mathbf{P}) = \text{tr}((\mathbf{P} - \mathbf{I}) \Sigma_{\mathbf{X}, \mathbf{X}} (\mathbf{P} - \mathbf{I})^\top) = \text{tr}((\mathbf{P} - \mathbf{I})(\mathbf{P} - \mathbf{I})^\top) = \|\mathbf{P} - \mathbf{I}\|_F^2. \quad (47)$$

The problem reduces to minimizing $\|\mathbf{P} - \mathbf{I}\|_F^2$ subject to constraint $\mathbf{P} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}$. Writing $\mathbf{P}$ by its rows $\mathbf{p}_i^\top$, the constraint forces each row to satisfy $\mathbf{p}_i^\top \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} = \mathbf{0}^\top$, each $\mathbf{p}_i$ lies in the orthogonal complement of $V \triangleq \text{colsp}(\Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}})$. The Frobenius objective decouples row-wise:

$$\|\mathbf{P} - \mathbf{I}\|_F^2 = \sum_i \|\mathbf{p}_i - \mathbf{e}_i\|^2, \quad (48)$$

so the optimum is found by independently projecting each $\mathbf{e}_i$ onto $V^\perp$: $\mathbf{p}_i^* = \Pi_{V^\perp} \mathbf{e}_i$. Stacking the rows using $\Pi_{V^\perp}^\top = \Pi_{V^\perp}$ yields $\mathbf{P}^* = \Pi_{V^\perp}$. By Hint 2 applied to $\mathbf{A} = \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}$:

$$\mathbf{P}^* = \Pi_{V^\perp} = \mathbf{I} - \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}^+. \quad (49)$$

i) Extension to Arbitrary Mean and Covariance via Whitening

Drop the simplifying assumptions and allow $\mathbb{E}[\mathbf{X}] \neq \mathbf{0}$ and $\Sigma_{\mathbf{X}, \mathbf{X}} \neq \mathbf{I}$. Define the whitening matrix $\mathbf{W} \triangleq (\Sigma_{\mathbf{X}, \mathbf{X}}^{1/2})^+$.

Whitening the representation. Let $\mathbf{Y} \triangleq \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}])$. Then

$$\mathbb{E}[\mathbf{Y}] = \mathbf{W}(\mathbb{E}[\mathbf{X}] - \mathbb{E}[\mathbf{X}]) = \mathbf{0}, \quad \Sigma_{\mathbf{Y}, \mathbf{Y}} = \mathbf{W} \Sigma_{\mathbf{X}, \mathbf{X}} \mathbf{W}^\top = \mathbf{I} \quad (50)$$

on the support of $\mathbf{X} - \mathbb{E}[\mathbf{X}]$. By translation invariance and linearity of covariance in its first argument:

$$\Sigma_{\mathbf{Y}, \tilde{\mathbf{Z}}} = \text{Cov}(\mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]), \tilde{\mathbf{Z}}) = \mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}. \quad (51)$$

Applying part (h) in the whitened space. Since $\mathbf{Y}$ now satisfies the assumptions of part (h), the optimal guarding projection in whitened coordinates is

$$\mathbf{P}_{\mathbf{Y}}^* = \mathbf{I} - \Sigma_{\mathbf{Y}, \tilde{\mathbf{Z}}} \Sigma_{\mathbf{Y}, \tilde{\mathbf{Z}}}^+ = \mathbf{I} - \Pi_{\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}}, \quad (52)$$

so the guarded whitened representation is $\hat{\mathbf{Y}} = \mathbf{P}_{\mathbf{Y}}^* \mathbf{Y}$.

Mapping back to the original space. Undo the whitening and centering by applying $\mathbf{W}^+ = \Sigma_{\mathbf{X}, \mathbf{X}}^{1/2}$ and adding back $\mathbb{E}[\mathbf{X}]$:

$$r(\mathbf{X}) = \mathbf{W}^+ \hat{\mathbf{Y}} + \mathbb{E}[\mathbf{X}] \quad (53)$$

$$= \mathbf{W}^+ \left(\mathbf{I} - \Pi_{\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}} \right) \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]) + \mathbb{E}[\mathbf{X}] \quad (54)$$

$$= \mathbf{W}^+ \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]) - \mathbf{W}^+ \Pi_{\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}} \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]) + \mathbb{E}[\mathbf{X}]. \quad (55)$$

The product $\mathbf{W}^+ \mathbf{W} = \Sigma_{\mathbf{X}, \mathbf{X}}^{1/2} (\Sigma_{\mathbf{X}, \mathbf{X}}^{1/2})^+$ is the orthogonal projection onto $\text{colsp}(\Sigma_{\mathbf{X}, \mathbf{X}}^{1/2})$, which contains $\mathbf{X} - \mathbb{E}[\mathbf{X}]$ almost surely, so $\mathbf{W}^+ \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]) = \mathbf{X} - \mathbb{E}[\mathbf{X}]$. The means cancel, giving the final form:

$$r(\mathbf{X}) = \mathbf{X} - \mathbf{W}^+ \Pi_{\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}} \mathbf{W}(\mathbf{X} - \mathbb{E}[\mathbf{X}]), \quad (56)$$

where $\Pi_{\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}}} = (\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}})(\mathbf{W} \Sigma_{\mathbf{X}, \tilde{\mathbf{Z}}})^+$ is the orthogonal projection in the whitened space onto the subspace encoding concept information.

j) Colab Notebook

The printed PDF of the Colab notebook is included on the following pages.

LLM Assignment 1: Representation Surgery (Coding)

IMPORTANT: MAKE A COPY OF THIS NOTEBOOK AND WORK IN THERE.

Welcome to the coding part of the Representation Surgery question in Assignment 1. The goal of this portion is to give you a deeper intuition for how representation surgery works and its strengths/limitations via hands-on implementations.

It consists of 2 parts:

1. Implementing linear concept erasure as described in the assignment, and applying it to a synthetic dataset.
2. Applying concept erasure to erase the concept of "gender" from the BERT representations of a dataset of biographies.

In this assignment, you'll have a series of tasks. For each task you will need to write some code and produce plots and/or accuracy results. You will need to export the notebook as a pdf and attach it to your submission. Please make sure all deliverables for each task (e.g., plots and accuracy results) are clearly visible, as we will be grading you on these outputs.

```
!git clone https://github.com/rycolab/llm-representation-surgery.git
```

```
Cloning into 'llm-representation-surgery'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 34.59 MiB | 16.86 MiB/s, done.
```

```
# These are all the imports you should need
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.cross_decomposition import CCA
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from typing import Tuple
np.random.seed(42)
```

0. Constructing a synthetic dataset

First, we'll give you a synthetic dataset encoding a concept, i.e., the target labels.

```
n, d, k = 2048, 128, 2
X, Z = make_classification(
    n_samples=n,
    n_informative=k,
    n_features=d,
    n_classes=k,
    random_state=1,
)
print(f"Shape of X: {X.shape}, Shape of Z: {Z.shape}")
```

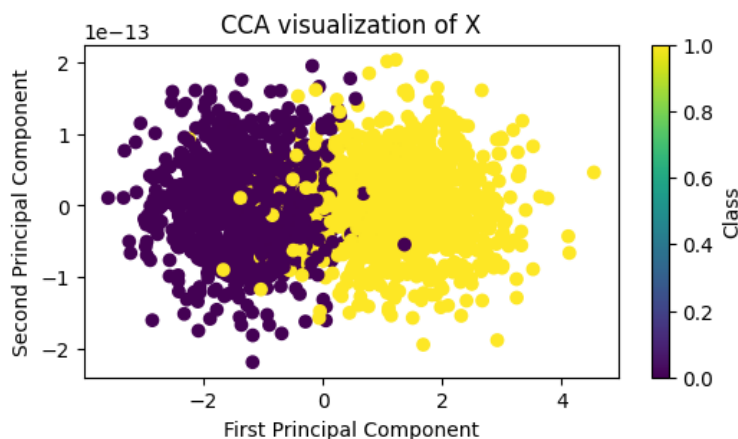
```
Shape of X: (2048, 128), Shape of Z: (2048,)
```

What does this dataset look like? Visualizing 128-dimensional data is hard, so we'll reduce the dimensionality to 2 using CCA and plot the result.

```
def plot_cca(X, Z):
    """
    Use Canonical Component Analysis to visualize the dataset X and label Z.
    """
    Z_one_hot = np.zeros((len(Z), len(set(Z))))
    Z_one_hot[np.arange(len(Z)), Z] = 1
    cca = CCA(n_components=2)
    X_cca = cca.fit_transform(X, y=Z_one_hot)[0]
    plt.figure(figsize=(6, 3))
    plt.scatter(X_cca[:, 0], X_cca[:, 1], c=Z, cmap='viridis')
    plt.colorbar(label='Class')
```

```
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('CCA visualization of X')
plt.show()
```

```
plot_cca(X, Z)
```



As you can see, in these two components, the two concepts are fairly separable.

Task 0.1: Implement a logistic regression classifier predict the concept Z from X .

FOR THE SUBMISSION: Print the accuracy your classifier scores on the data.

```
def logreg(X: np.ndarray, Z: np.ndarray) -> float:
    """
    Train a logistic regression classifier to predict concept Z from X.

    Args:
        X - the data
        Z - the concept/label

    Return:
        Accuracy in predicting Z from X.
    """
    clf = LogisticRegression(max_iter=1000)
    clf.fit(X, Z)
    return clf.score(X, Z)

print(f"Accuracy before erasure: {logreg(X, Z):.3f}")

Accuracy before erasure: 0.941
```

1. Implementing linear concept erasure

Task 1.1: Validate that the resulting whitening and unwhitening matrices satisfy their definition and goals.

That is, verify (a) $W W^T = I$ and (b) $\text{cov}(XW) = I$.

FOR THE SUBMISSION: Include your plots showing the whitening matrices.

```
def compute_W(X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """
    Compute the whitening and unwhitening matrix for the data using the ZCA whitening method
    (see Appendix A of https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf for more details)
    We give this to you as a numerically stable implementation.
    """
    sigma = np.cov(X.T)

    L, V = np.linalg.eigh(sigma)

    # Assuming PSD; account for numerical error
```

```

L = L.clip(0.0)

# Threshold used by torch.linalg.pinv
mask = L > (L[-1] * sigma.shape[-1] * np.finfo(float).eps)

with np.errstate(divide='ignore'):
    W = V * np.where(mask, 1/np.sqrt(L), 0.0) @ V.T
    W_inv = V * np.where(mask, np.sqrt(L), 0.0) @ V.T

return W, W_inv

def heatmap(arr, title):
    plt.figure(figsize=(10, 8))
    sns.heatmap(arr, cmap='viridis')
    plt.title(title)
    plt.tight_layout()
    plt.show()

W, W_inv = compute_W(X)

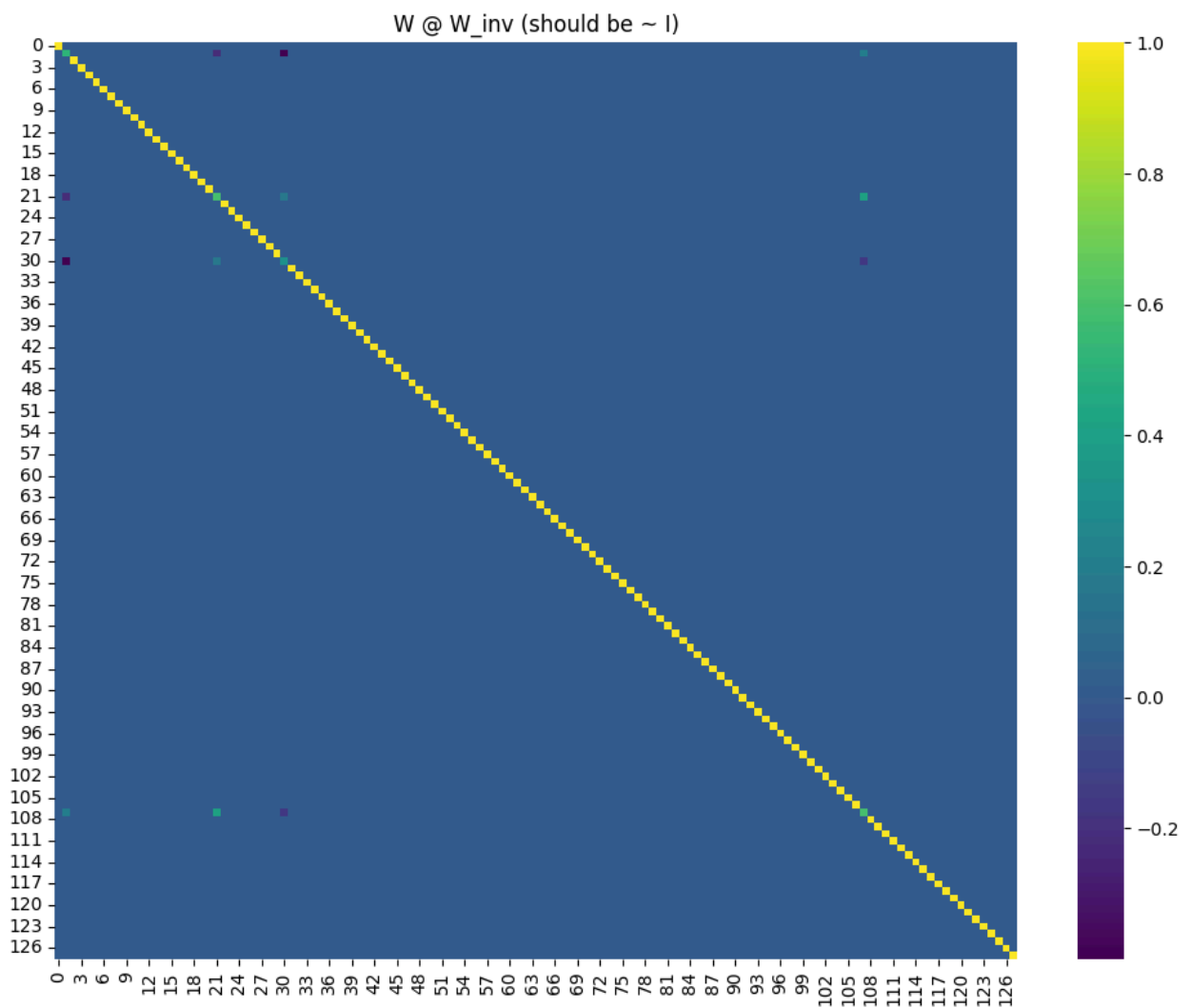
```

```

def verify_W_W_inv(W, W_inv):
    """
    Plot a heatmap of W @ W_inv to verify it satisfies the definition of whitening.
    """
    heatmap(W @ W_inv, title="W @ W_inv (should be ~ I)")

verify_W_W_inv(W, W_inv)

```



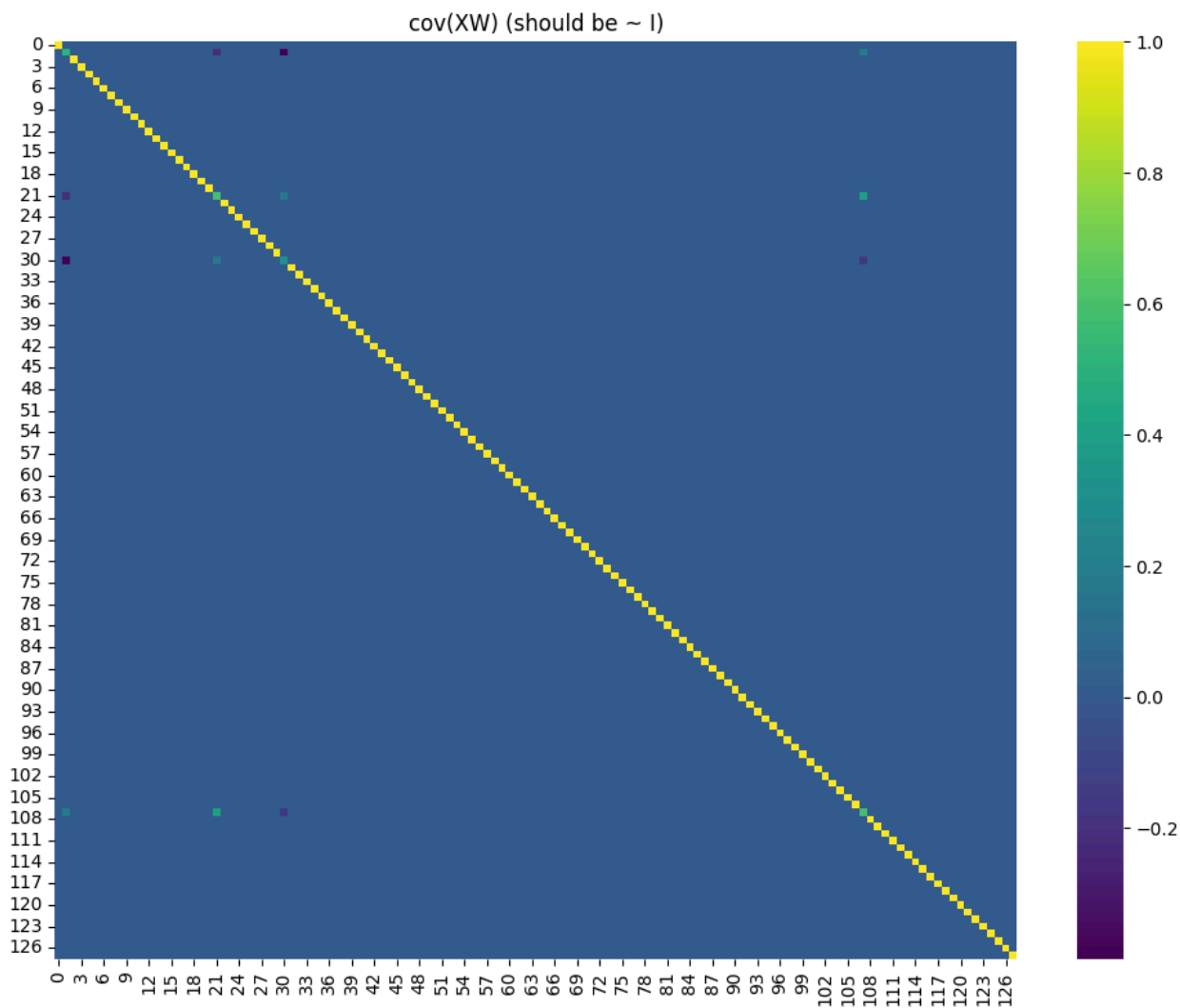
```

def verify_cov_XW(X, W):
    """
    Plot a heatmap of the covariance of XW to verify it satisfies the definition of whitening.
    """
    cov_XW = np.cov((X @ W).T)

```

```
heatmap(cov_XW, title="cov(XW) (should be ~ I)")

verify_cov_XW(X, W)
```



Task 1.2: Implement linear concept erasure

Use the equations derived from the written part of the assignment.

FOR THE WRITEUP: Include your plot and accuracy of the embeddings before erasure and after erasure.

```
def erase(X: np.ndarray, Z: np.ndarray) -> np.ndarray:
    """
    Implement linear concept erasure.

    Args:
        X - the data, shape (n, d)
        Z - the concept/label, shape (n,)

    Return:
        The data with the concept erased. Shape (n, d).
    """
    n = X.shape[0]
    k = len(np.unique(Z))

    Z_one_hot = np.zeros((n, k))
    Z_one_hot[np.arange(n), Z] = 1

    mean_X = X.mean(axis=0)
    X_centered = X - mean_X
    Z_centered = Z_one_hot - Z_one_hot.mean(axis=0)

    Sigma_XZ = (X_centered.T @ Z_centered) / n
```

```

W, W_inv = compute_W(X)

WSigma = W @ Sigma_XZ
Pi = WSigma @ np.linalg.pinv(WSigma)

return X - X_centered @ W @ Pi @ W_inv

erased_X = erase(X, Z)

```

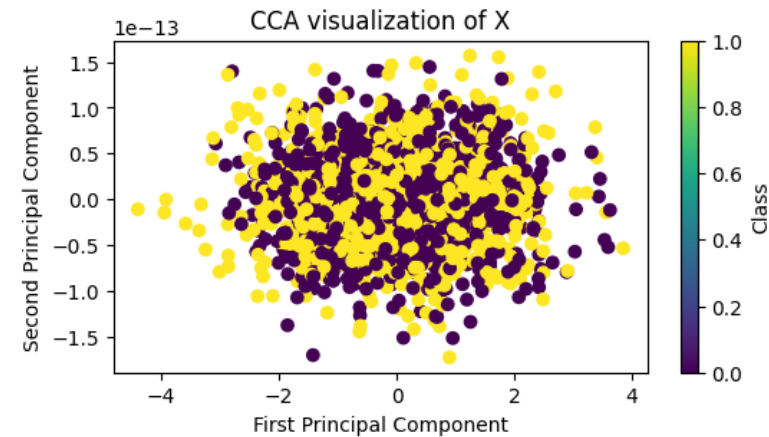
Now, let's validate that the concept has been erased.

```

plot_cca(erased_X, Z)
print(f"Accuracy after erasure: {logreg(erased_X, Z)}")

```

/usr/local/lib/python3.12/dist-packages/sklearn/cross_decomposition/_pls.py:113: ConvergenceWarning: Maximum number of iterations reached, ConvergenceWarning)



2. Erasing gender bias

In this portion, we'll apply the same method to a real dataset of word embeddings of biography data. Your goal is to (a) construct sentence embeddings and (b) erase the gender concept from these embeddings. Use the

hard_text_untokenized column for your embeddings.

```

df = pd.read_csv("llm-representation-surgery/data/biographies.csv")
df.head()

```

	g	p	text	start	hard_text	hard_text_untokenized	text_without_gender
0	f	psychologist	Brandi Smith is a registered psychologist in t...	139	Ms. Smith is experienced in providing psychoth...	Ms. Smith is experienced in providing psychoth...	_ _ is experienced in providing psychotherapy...
1	m	software_engineer	Duarte Nunes is a Software Engineer working on...	56	He has a background in concurrent programming ...	He has a background in concurrent programming,...	_ has a background in concurrent programming, ...
2	f	software_engineer	Lauren Ellsworth is a Senior Software Engineer...	139	In her years of development , Lauren has contr...	In her years of development, Lauren has contri...	In _ years of development, _ has contributed t...
3	f	professor	Gillian McCann is an Associate Professor in th...	124	She holds a doctorate in Religious Studies in ...	She holds a doctorate in Religious Studies in ...	_ holds a doctorate in Religious Studies in th...
4	m	professor	BIOGRAPHY: Jason Del Gandio (PhD)	113	His area of expertise is the theory and	His area of expertise is the theory and practi...	_ area of expertise is the theory and practice...

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Task 2.1: Compute embeddings and use the `erase` function defined above to erase the gender concept from the embeddings.

FOR THE FOR THE SUBMISSION: Include: (a) CCA plot of the embeddings before erasure, (b) CCA plot of the embeddings after erasure, (c) logistic regression accuracy before erasure, and (d) logistic regression accuracy after erasure.

```

from transformers import AutoModel, AutoTokenizer, logging
import torch
import warnings
import os

# Suppress warnings and logging messages
warnings.filterwarnings("ignore")
logging.set_verbosity_error()
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"

def extract_gender_labels(df: pd.DataFrame) -> np.ndarray:
    """
    Extract the gender labels from the dataframe.
    """
    return LabelEncoder().fit_transform(df["g"])

def compute_embeddings(df: pd.DataFrame) -> np.ndarray:
    """
    Compute the embeddings using a pretrained model for the "hard_text_untokenized" column of the given df.
    Feel free to experiment with different models; a good starting point is the classic BERT model, "google/bert-base-uncased".
    Also feel free to use any libraries which would be helpful, such as `transformers` or `sentence-transformers`.
    """
    model_name = "google-bert/bert-base-uncased"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name)
    device = (
        "cuda" if torch.cuda.is_available()
        else ("mps" if torch.backends.mps.is_available() else "cpu")
    )
    model.to(device).eval()

    texts = df["hard_text_untokenized"].astype(str).tolist()
    batch_size = 32
    embeddings = []
    with torch.no_grad():
        for i in range(0, len(texts), batch_size):
            batch = texts[i : i + batch_size]
            enc = tokenizer(
                batch,
                padding=True,
                truncation=True,
                max_length=128,
                return_tensors="pt",
            ).to(device)
            out = model(**enc).last_hidden_state # (B, T, H)
            mask = enc["attention_mask"].unsqueeze(-1) # (B, T, 1)
            pooled = (out * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
            embeddings.append(pooled.cpu().numpy())
    return np.vstack(embeddings)

def erase_gender(embeddings: np.ndarray, gender_labels: np.ndarray) -> np.ndarray:
    """
    This function should do 3 things:
    1. Erase the gender concept from the embeddings and return these new embeddings.
    2. Visualize the embeddings before erasure and print the accuracy before erasure.
    3. Visualize the embeddings after erasure and print the accuracy after erasure.

    Args:
        embeddings: The embeddings to erase the gender concept from.
        gender_labels: The gender labels for the embeddings.

    Returns:
        The embeddings with the gender concept erased.
    """
    print("---- Before erasure ----")

```

```

plot_cca(embeddings, gender_labels)
print(
    f"Linear accuracy before erasure: "
    f"{logreg(embeddings, gender_labels):.3f}"
)

erased = erase(embeddings, gender_labels)

print("--- After erasure ---")
plot_cca(erased, gender_labels)
print(
    f"Linear accuracy after erasure: "
    f"{logreg(erased, gender_labels):.3f}"
)
return erased

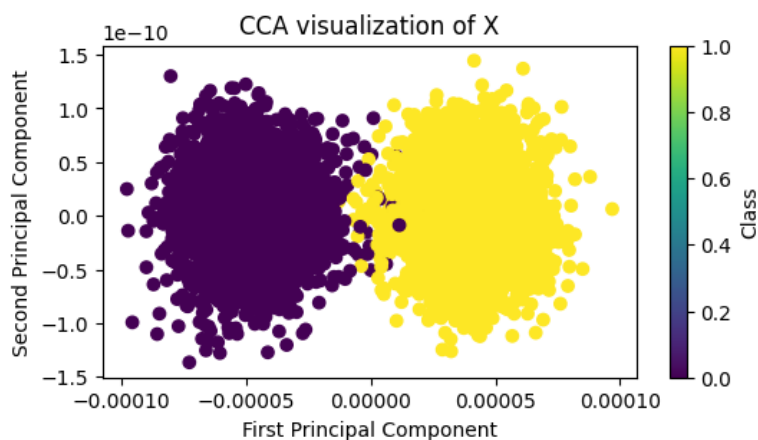
gender_labels = extract_gender_labels(df)
embeddings = compute_embeddings(df)
embeddings_erased = erase_gender(embeddings, gender_labels)

```

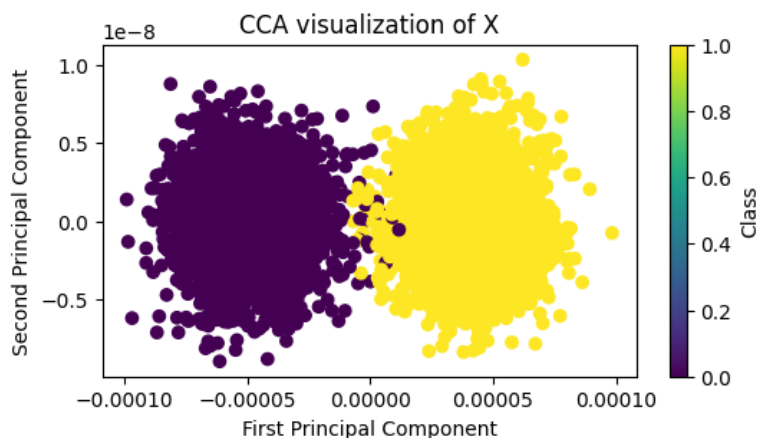
Loading weights: 100%

199/199 [00:00<00:00, 702.18it/s, Materializing param=pooler.dense.weight]

--- Before erasure ---



--- After erasure ---



Task 2.2: Verifying minimally-destructive edits.

One key aspect of concept erasure is we want the edits to be *minimal*, i.e., they should not destroy other aspects of the representations. To test the degree to which the representation is preserved, use the profession as a proxy. Show that the profession information is preserved within the embeddings by computing the accuracy (a) before erasure and (b) after erasure.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```

def visualize_profession(df: pd.DataFrame) -> np.ndarray:
    """
    Visualize the label distribution of professions, sorted from highest to lowest.
    """

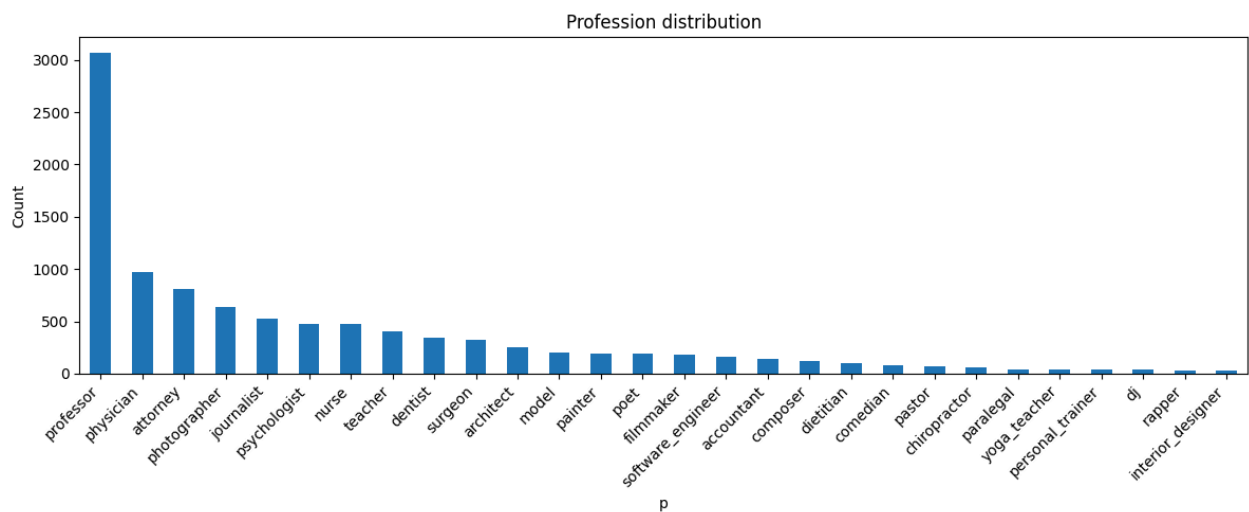
```

```

counts = df["p"].value_counts()
plt.figure(figsize=(12, 5))
counts.plot(kind="bar")
plt.title("Profession distribution")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

```
visualize_profession(df)
```



```

def verify_profession_intact(embeddings: np.ndarray, embeddings_erased: np.ndarray, df: pd.DataFrame) -
    """
    Verify the profession information is preserved within the embeddings.
    """
    professions = LabelEncoder().fit_transform(df["p"])
    acc_before = logreg(embeddings, professions)
    acc_after = logreg(embeddings_erased, professions)
    print(f"Profession accuracy before erasure: {acc_before:.3f}")
    print(f"Profession accuracy after erasure: {acc_after:.3f}")

    verify_profession_intact(embeddings, embeddings_erased, df)

```

```

Profession accuracy before erasure: 0.926
Profession accuracy after erasure: 0.923

```

Task 2.3: Show that using a nonlinear classifier, you can recover the concept from the erased embeddings.

While we've successfully removed gender from being *linearly encoded* in the embeddings, they can still be extracted using other models. Your goal is to extract the concept using some other model.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```

from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

def nonlinear_classifier(embeddings: np.ndarray, gender_labels: np.ndarray) -> float:
    """
    Train a nonlinear classifier on the embeddings to predict the gender labels and return the accuracy
    """
    X_tr, X_te, y_tr, y_te = train_test_split(
        embeddings,
        gender_labels,
        test_size=0.3,
        random_state=42,
        stratify=gender_labels,
    )
    clf = MLPClassifier(
        hidden_layer_sizes=(128, 64),
        activation="relu",
        max_iter=500,
    )

```

```
        random_state=42,
    )
    clf.fit(X_tr, y_tr)
    return clf.score(X_te, y_te)

print(
    f"Nonlinear test accuracy before erasure: "
    f"{nonlinear_classifier(embeddings, gender_labels):.3f}"
)
print(
    f"Nonlinear test accuracy after erasure: "
    f"{nonlinear_classifier(embeddings_erased, gender_labels):.3f}"
)
```

```
Nonlinear test accuracy before erasure: 0.994
Nonlinear test accuracy after erasure: 0.920
```

Question 2: Properties of Finite-state Language Models

a) The Expected String Length

To compute the expected string length μ , we must translate the probability distribution over strings into matrix operations. $\mu \triangleq \sum_{\mathbf{y} \in \Sigma^*} p(\mathbf{y})|\mathbf{y}|$.

In a probabilistic finite-state automaton (PFSA) $\mathcal{A} = (\Sigma, Q, \delta, \lambda, \rho)$, we can group all strings by their length k . The probability of the model generating *any* string of length k is given by the sum of the weights of all paths of length k . Using the transition matrix $\mathbf{T}$, the initial state probabilities λ , and the stopping probabilities ρ , the total probability for all strings of length k is given by $\lambda^\top \mathbf{T}^k \rho$.

$$\mu = \sum_{k=0}^{\infty} k (\lambda^\top \mathbf{T}^k \rho) = \lambda^\top \left(\sum_{k=0}^{\infty} k \mathbf{T}^k \right) \rho \quad (57)$$

To compute this in $\mathcal{O}(|Q|^3)$, we can call the `allsum` algorithm as a black box. The `allsum` algorithm evaluates the sum of weights for all possible paths, computing $\sum_{k=0}^{\infty} \mathbf{W}^k = (\mathbf{I} - \mathbf{W})^{-1}$ for a given weight matrix $\mathbf{W}$.

To make our equation match the `allsum` formulation, we can construct a new augmented automaton using a block matrix structure.

Define a modified PFSA over an expanded state space Q' where each original state is duplicated, meaning $|Q'| = 2|Q|$. This allows the transition matrix to accumulate the path length k as it is raised to higher powers.

Define a new transition block matrix $\mathbf{M} \in \mathbb{R}^{2|Q| \times 2|Q|}$, a new initial state vector $\lambda' \in \mathbb{R}^{2|Q|}$, and a new stopping vector $\rho' \in \mathbb{R}^{2|Q|}$ as follows:

$$\mathbf{M} = \begin{pmatrix} \mathbf{T} & \mathbf{T} \\ \mathbf{0} & \mathbf{T} \end{pmatrix}, \quad \lambda' = \begin{pmatrix} \lambda \\ \mathbf{0} \end{pmatrix}, \quad \rho' = \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (58)$$

When we raise the block matrix $\mathbf{M}$ to the power of k , the diagonal blocks remain $\mathbf{T}^k$, while the upper-right block accumulates the sum of the products. For any integer $k \geq 0$, we have:

$$\mathbf{M}^k = \begin{pmatrix} \mathbf{T}^k & k \mathbf{T}^k \\ \mathbf{0} & \mathbf{T}^k \end{pmatrix} \quad (59)$$

If we apply the `allsum` algorithm to this new augmented automaton, it computes the sum of the weights over all paths of all lengths:

$$\text{allsum}(\mathbf{M}, \lambda', \rho') = \sum_{k=0}^{\infty} \lambda'^\top \mathbf{M}^k \rho' \quad (60)$$

Substituting block definitions into this equation shows the length coefficient:

$$\lambda'^\top \mathbf{M}^k \rho' = (\lambda^\top \quad \mathbf{0}^\top) \begin{pmatrix} \mathbf{T}^k & k \mathbf{T}^k \\ \mathbf{0} & \mathbf{T}^k \end{pmatrix} \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (61)$$

$$= (\lambda^\top \mathbf{T}^k \quad \lambda^\top (k \mathbf{T}^k)) \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (62)$$

$$= \lambda^\top k \mathbf{T}^k \rho \quad (63)$$

Summing this extraction over all $k \in [0, \infty)$ yields the expected length μ :

$$\sum_{k=0}^{\infty} \lambda'^{\top} \mathbf{M}^k \rho' = \sum_{k=0}^{\infty} \lambda^{\top} k \mathbf{T}^k \rho = \mu \quad (64)$$

The Algorithm

1. Construct the block matrix $\mathbf{M}$ alongside vectors λ' and ρ' .
2. Call the standard `allsum` function, passing $(\mathbf{M}, \lambda', \rho')$ as the arguments.
3. Return the scalar output as μ .

Proof of Time Complexity

- Constructing the augmented matrix $\mathbf{M}$ and the vectors involves allocating memory and copying the elements of $\mathbf{T}$, which takes $\mathcal{O}(|Q|^2)$ time.
- The `allsum` algorithm evaluates the infinite sum by calculating the inverse $(\mathbf{I} - \mathbf{M})^{-1}$, or by utilizing Gaussian elimination to solve the equivalent linear system. Matrix inversion and solving linear systems both inherently have a cubic time complexity relative to the size of the matrix.
- Since our augmented matrix $\mathbf{M}$ has dimensions $2|Q| \times 2|Q|$, the `allsum` algorithm will execute in $\mathcal{O}((2|Q|)^3)$ time.
- Because constants can be dropped in Big-O notation, $\mathcal{O}(8|Q|^3)$ simplifies directly to $\mathcal{O}(|Q|^3)$.

The $\mathcal{O}(|Q|^2)$ setup time is strictly bounded and dominated by the $\mathcal{O}(|Q|^3)$ `allsum` operation. Thus, the total overall runtime of the algorithm is proven to be $\mathcal{O}(|Q|^3)$.

Alternative Implementation To elaborate further on the inherent properties of finite-state language models, we can also bypass the block matrix construction to yield a slightly more memory-efficient algorithm that still meets the $\mathcal{O}(|Q|^3)$ requirement. We know from geometric series properties of matrices that $\sum_{k=0}^{\infty} \mathbf{T}^k = (\mathbf{I} - \mathbf{T})^{-1}$. If we evaluate the infinite sum $\sum_{k=0}^{\infty} k \mathbf{T}^k$ by factoring out $\mathbf{T}$ and rewriting it as a matrix derivative, we arrive at:

$$\sum_{k=0}^{\infty} k \mathbf{T}^k = \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2} = (\mathbf{I} - \mathbf{T})^{-1} \mathbf{T} (\mathbf{I} - \mathbf{T})^{-1} \quad (65)$$

This means the mean string length can also be written purely algebraically as $\mu = \lambda^{\top} (\mathbf{I} - \mathbf{T})^{-1} \mathbf{T} (\mathbf{I} - \mathbf{T})^{-1} \rho$. We can compute this directly using a sequence of linear algebraic operations:

1. **Solve** the linear system $(\mathbf{I} - \mathbf{T})\mathbf{v}_1 = \rho$ to find $\mathbf{v}_1$. Time complexity: $\mathcal{O}(|Q|^3)$.
2. **Multiply** the transition matrix to find $\mathbf{v}_2 = \mathbf{T}\mathbf{v}_1$. Time complexity: $\mathcal{O}(|Q|^2)$.
3. **Solve** the second linear system $(\mathbf{I} - \mathbf{T})\mathbf{v}_3 = \mathbf{v}_2$ to find $\mathbf{v}_3$. Time complexity: $\mathcal{O}(|Q|^3)$.
4. **Compute** the final dot product $\mu = \lambda^{\top} \mathbf{v}_3$. Time complexity: $\mathcal{O}(|Q|)$.

The sum of these sequential execution times is $\mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^2) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|)$, which perfectly asymptotes to the required $\mathcal{O}(|Q|^3)$ runtime. Both methods are formally correct, though the block matrix approach directly fulfills the hint to deploy the existing `allsum` operation as an undisturbed black box.

b) The Variance of String Lengths

The variance of the string length is defined as $\sigma^2 = \sum_{\mathbf{y} \in \Sigma^*} p(\mathbf{y})(|\mathbf{y}| - \mu)^2$. Using the linearity of expectation:

$$\sigma^2 = \mathbb{E}[|\mathbf{y}|^2] - \mu^2 \quad (66)$$

Sum over all path lengths k to compute the second moment $\mathbb{E}[|\mathbf{y}|^2]$:

$$\mathbb{E}[|\mathbf{y}|^2] = \sum_{k=0}^{\infty} k^2 (\lambda^\top \mathbf{T}^k \rho) = \lambda^\top \left(\sum_{k=0}^{\infty} k^2 \mathbf{T}^k \right) \rho \quad (67)$$

Decompose the k^2 into $k(k-1) + k$ to separate the sum:

$$\sum_{k=0}^{\infty} k^2 \mathbf{T}^k = \sum_{k=0}^{\infty} k(k-1) \mathbf{T}^k + \sum_{k=0}^{\infty} k \mathbf{T}^k \quad (68)$$

These sums map to the derivatives of the geometric matrix series $\sum_{k=0}^{\infty} \mathbf{T}^k = (\mathbf{I} - \mathbf{T})^{-1}$:

1. The first gives the sum for k : $\sum_{k=0}^{\infty} k \mathbf{T}^k = \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2}$.
2. The second gives the sum for $k(k-1)$: $\sum_{k=0}^{\infty} k(k-1) \mathbf{T}^k = 2\mathbf{T}^2(\mathbf{I} - \mathbf{T})^{-3}$.

$$\sum_{k=0}^{\infty} k^2 \mathbf{T}^k = 2\mathbf{T}^2(\mathbf{I} - \mathbf{T})^{-3} + \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2} \quad (69)$$

$$= (\mathbf{I} - \mathbf{T})^{-3} (2\mathbf{T}^2 + \mathbf{T}(\mathbf{I} - \mathbf{T})) = (\mathbf{I} - \mathbf{T})^{-3} (\mathbf{T}^2 + \mathbf{T}) \quad (70)$$

Thus, the second moment is $\mathbb{E}[|\mathbf{y}|^2] = \lambda^\top (\mathbf{T}^2 + \mathbf{T})(\mathbf{I} - \mathbf{T})^{-3} \rho$. Finally, the variance is $\sigma^2 = \lambda^\top (\mathbf{T}^2 + \mathbf{T})(\mathbf{I} - \mathbf{T})^{-3} \rho - \mu^2$.

Algorithm

1. **Compute μ :** Use the algorithm in (a) to find the mean string length.
2. **Initialize:** Define the base vector $\mathbf{v}_0 = \rho$.
3. **Solve First System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_1 = \mathbf{v}_0$ for $\mathbf{v}_1$. (This computes $(\mathbf{I} - \mathbf{T})^{-1}\rho$).
4. **Solve Second System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_2 = \mathbf{v}_1$ for $\mathbf{v}_2$. (This computes $(\mathbf{I} - \mathbf{T})^{-2}\rho$).
5. **Solve Third System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_3 = \mathbf{v}_2$ for $\mathbf{v}_3$. (This computes $(\mathbf{I} - \mathbf{T})^{-3}\rho$).
6. **Compute Polynomial Matrix:** Calculate the matrix $\mathbf{U} = \mathbf{T}^2 + \mathbf{T}$.
7. **Compute Vector:** Calculate the product $\mathbf{w} = \mathbf{U} \mathbf{v}_3$.
8. **Compute Second Moment:** Calculate the dot product $\mathbb{E}[|\mathbf{y}|^2] = \lambda^\top \mathbf{w}$.
9. **Compute Variance:** Return the variance as $\sigma^2 = \mathbb{E}[|\mathbf{y}|^2] - \mu^2$.

Proof of Time Complexity

- **Step 1:** Computing the expected length μ was proven in (a) to take $\mathcal{O}(|Q|^3)$.
- **Steps 3, 4, 5:** Solving a linear system of size $|Q| \times |Q|$ has a worst-case time complexity of $\mathcal{O}(|Q|^3)$. We solve three linear systems sequentially making the time taken is $3 \times \mathcal{O}(|Q|^3)$, which asymptotically is $\mathcal{O}(|Q|^3)$.
- **Step 6:** Multiplying two square matrices of size $|Q| \times |Q|$ to calculate $\mathbf{T}^2$ requires $\mathcal{O}(|Q|^3)$ time. Adding $\mathbf{T}$ requires $\mathcal{O}(|Q|^2)$ additions.

- **Step 7:** Multiplying the $|Q| \times |Q|$ matrix $\mathbf{U}$ by the $|Q| \times 1$ vector $\mathbf{v}_3$ takes $\mathcal{O}(|Q|^2)$.
- **Step 8:** The dot product of two vectors of size $|Q|$ executes in $\mathcal{O}(|Q|)$.
- **Step 9:** A single scalar subtraction operation happens in $\mathcal{O}(1)$ time.

Total sum of the time complexity:

$$\mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^2) + \mathcal{O}(|Q|) + \mathcal{O}(1) \quad (71)$$

The highest order term dominates the growth rate, so the algorithm runs in time $\mathcal{O}(|Q|^3)$.

c) Entropy of a Deterministic Finite-State Language Model

Because the automaton $\mathcal{A}$ is deterministic, any string $\mathbf{y}$ corresponds to one unique path of transitions through the automaton from a given starting state.

- Let Π_q be a random variable representing the paths in $\mathcal{A}$ starting in state q .
- Let $\mathbf{H}(\Pi_q)$ denote the entropy of the distribution of paths originating from state q .

Decompose a path Π_q into its first step and the remainder of the path.

- Let τ_q be a random variable representing the outgoing transition from q or its termination.
- Because $\mathcal{A}$ is deterministic, the destination state q' is uniquely determined by τ_q . Consequently, the path Π_q is in bijection with the pair $(\tau_q, \Pi_{q'})$, so their entropies coincide: $\mathbf{H}(\Pi_q) = \mathbf{H}(\tau_q, \Pi_{q'})$.
- By the chain rule of entropy, $\mathbf{H}(x, y) = \mathbf{H}(x) + \mathbf{H}(y \mid x)$. Applying this to the joint variable $(\tau_q, \Pi_{q'})$ yields:

$$\mathbf{H}(\Pi_q) = \mathbf{H}(\tau_q, \Pi_{q'}) = \mathbf{H}(\tau_q) + \mathbf{H}(\Pi_{q'} \mid \tau_q) \quad (72)$$

Evaluate the entropy of the first step, $\mathbf{H}(\tau_q)$. The probability distribution for τ_q is defined taking the transition $q \xrightarrow{y/w} q'$ with w , and taking the termination with ρ_q . The entropy of this single step is the negative sum of $p \log p$ for all possible outcomes:

$$\mathbf{H}(\tau_q) = - \sum_{q \xrightarrow{y/w} q'} w \log w - \rho_q \log \rho_q = \boldsymbol{\xi}_q \quad (73)$$

Evaluate the conditional entropy of the remaining path, $\mathbf{H}(\Pi_{q'} \mid \tau_q)$ represents the expected entropy of the rest of the path, given the first step τ_q . If τ_q is termination with probability ρ_q , the sequence ends, and the entropy of the remaining empty path is 0. If τ_q is a transition to state q' with probability w , the remaining path originates from q' , so its entropy is $\mathbf{H}(\Pi_{q'})$. Summing these expectations over all outgoing transitions gives:

$$\mathbf{H}(\Pi_{q'} \mid \tau_q) = \sum_{q \xrightarrow{y/w} q'} w \mathbf{H}(\Pi_{q'}) \quad (74)$$

Group this sum by the destination state q' . By definition, the transition matrix entry $\mathbf{T}_{q,q'}$ is the sum of the weights w of all transitions from q to q' . Therefore:

$$\mathbf{H}(\Pi_{q'} \mid \tau_q) = \sum_{q' \in Q} \mathbf{T}_{q,q'} \mathbf{H}(\Pi_{q'}) \quad (75)$$

Construct the matrix equation by substituting the results:

$$\mathbf{H}(\Pi_q) = \boldsymbol{\xi}_q + \sum_{q' \in Q} \mathbf{T}_{q,q'} \mathbf{H}(\Pi_{q'}) \quad (76)$$

To solve, let $\mathbf{H} \in \mathbb{R}^{|Q|}$ be a column vector where the q -th entry is $H(\Pi_q)$. Represent the system of equations for all states as a single matrix equation:

$$\mathbf{H} = \boldsymbol{\xi} + \mathbf{T} \mathbf{H} \quad (77)$$

Solve for the path entropy vector $\mathbf{H}$ by isolating $\mathbf{H}$ using algebraic matrix operations:

$$\mathbf{H} - \mathbf{T} \mathbf{H} = \boldsymbol{\xi} = (\mathbf{I} - \mathbf{T}) \mathbf{H} \quad (78)$$

Because all eigenvalues of the transition matrix $\mathbf{T}$ have a magnitude less than 1, the matrix $(\mathbf{I} - \mathbf{T})$ is guaranteed to be invertible:

$$\mathbf{H} = (\mathbf{I} - \mathbf{T})^{-1} \boldsymbol{\xi} \quad (79)$$

The overall entropy of the language model is the entropy of the path distribution. Since $\mathcal{A}$ is deterministic, distinct paths produce distinct strings, so $H(p) = H(\Pi)$. Decompose a path into its starting state q_0 and the remainder; by the chain rule of entropy,

$$H(\Pi) = H(q_0) + H(\Pi \mid q_0) = H(\lambda) + \mathbb{E}_{q_0 \sim \lambda}[H(\Pi_{q_0})] = H(\lambda) + \lambda^\top \mathbf{H} \quad (80)$$

where $H(\lambda) = -\sum_{q \in Q} \lambda_q \log \lambda_q$ is the entropy of the initial-state distribution.

Substitute the solved vector $\mathbf{H}$ into the final equation:

$$H(p) = H(\lambda) + \lambda^\top (\mathbf{I} - \mathbf{T})^{-1} \boldsymbol{\xi} \quad (81)$$

In the special case where $\mathcal{A}$ has a unique initial state, λ is a point mass and $H(\lambda) = 0$, recovering $H(p) = \lambda^\top (\mathbf{I} - \mathbf{T})^{-1} \boldsymbol{\xi}$.

Question 3: Efficiently Simulating Bounded Stacks with RNNs

- a) The language $D(2, 3)$ consists of brackets of $k = 2$ types with maximum nesting depth $m = 3$. Encode the unclosed opening brackets using a stack whose *rightmost* symbol is the top of the stack. $[\langle_1 \langle_2]$ transitions to $[\langle_1]$ after reading $\rangle_2$.
- Reading a closing bracket $\rangle_i$: pop the top of the stack if it equals $\langle_i$; otherwise (mismatch or empty stack), reject.
 - Reading an opening bracket $\langle_i$: push it on top, unless the stack already has $m = 3$ symbols, in which case the depth bound is exceeded and we reject.

	$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_1 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_2$
$\rangle_2$	$\emptyset$	reject	$\langle_2 \langle_1$
$\rangle_1$	reject	$\langle_1 \langle_1$	reject
$\langle_2$	$\langle_2 \langle_2$	reject	reject

Table 1: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

- b) Construct a DFA $\mathcal{A} = (\Sigma, Q, I, F, \delta)$ that recognizes $D(k, m)$ by encoding every legal stack configuration as a state. Let $\langle = \{\langle_1, \dots, \langle_k\}$ and $\rangle = \{\rangle_1, \dots, \rangle_k\}$.

Alphabet: $\Sigma = \langle \cup \rangle$, so $|\Sigma| = 2k$.

States: $Q = \{[\gamma] \mid \gamma \in \langle^{\leq m}\} \cup \{R\}$, where $[\gamma]$ corresponds to the stack of unclosed brackets γ with $|\gamma| \leq m$, and R is an absorbing rejecting state. $A \stackrel{\text{def}}{=} [\emptyset]$ for the empty-stack state.

Initial states: $I = \{A\}$ (start with an empty stack).

Final states: $F = \{A\}$ (only the empty stack accepts).

Transition $\delta : Q \times \Sigma \rightarrow Q$: For valid prefix $\gamma \in \langle^*$ with $[\gamma] \in Q$ and $i, j \in \{1, \dots, k\}$:

1. *Valid push* (if $|\gamma| < m$): $\delta([\gamma], \langle_i) = [\gamma \langle_i]$.
2. *Valid pop*: $\delta([\gamma \langle_i], \rangle_i) = [\gamma]$.
3. *Depth overflow* (if $|\gamma| = m$): $\delta([\gamma], \langle_i) = R$.
4. *Mismatched close* (if $i \neq j$): $\delta([\gamma \langle_j], \rangle_i) = R$.
5. *Underflow*: $\delta(A, \rangle_i) = R$.
6. *Sink*: $\delta(R, s) = R$ for every $s \in \Sigma$.

These cases specify δ on every $(Q \setminus \{R\}) \times \Sigma$ pair as well as on $\{R\} \times \Sigma$, so δ is deterministic.

Correctness A string y is in $D(k, m)$ iff (i) the brackets are balanced and (ii) the running depth $d(y_{\leq t}) \leq m$. The state $[\gamma_t]$ reached after reading $y_{\leq t}$ stores the multiset *and order* of currently unclosed brackets, so $|\gamma_t| = d(y_{\leq t})$. The transitions enforce the two conditions:

- Any depth violation triggers rule (3), sending $\mathcal{A}$ into the absorbing R .
- Any mismatched close (rule 4) or underflow (rule 5) likewise sends $\mathcal{A}$ to R .

Conversely, while $y_{\leq t} \in D(k, m)$ at every prefix, only rules (1) and (2) fire and the automaton stays inside $\{[\gamma] \mid \gamma \in \langle^{\leq m}\}$. Since $F = \{A\}$, the run accepts iff after reading the whole string the stack is empty, i.e. all opens were correctly matched. Hence $\mathcal{L}(\mathcal{A}) = D(k, m)$.

Complexity

$$|Q| = 1 + \sum_{j=0}^m k^j = 1 + \frac{k^{m+1} - 1}{k - 1} = \mathcal{O}(k^m).$$

Because $\mathcal{A}$ is a DFA, the number of transitions is

$$|\delta| = |Q| \cdot |\Sigma| = \mathcal{O}(k^m) \cdot 2k = \mathcal{O}(k^{m+1}).$$

- c) For $D(2, 3)$ we have $k = 2$ and $m = 3$, so each hidden state lives in $\mathbb{R}^{k \cdot m} = \mathbb{R}^6$. Writing $\mathbf{e}_1 = (1, 0)^\top$ and $\mathbf{e}_2 = (0, 1)^\top$ for the canonical basis vectors of $\mathbb{R}^2$ associated with $\langle 1$ and $\langle 2$, Eq. (27) places the top of the stack in the first k -block and pads unused bottom blocks with zeros. The empty stack maps to $h([\emptyset]) = \mathbf{0}_6$, and the initial embeddings of $\gamma_1, \gamma_2, \gamma_3$ are

$$\begin{aligned} h(\gamma_1) &= h([\langle 2]) = \begin{pmatrix} \mathbf{e}_2 \\ \mathbf{0}_2 \\ \mathbf{0}_2 \end{pmatrix} = (0, 1, 0, 0, 0, 0)^\top, \\ h(\gamma_2) &= h([\langle 1 \langle 1]) = \begin{pmatrix} \mathbf{e}_1 \\ \mathbf{e}_1 \\ \mathbf{e}_1 \end{pmatrix} = (1, 0, 1, 0, 1, 0)^\top, \\ h(\gamma_3) &= h([\langle 2 \langle 1 \langle 2]) = \begin{pmatrix} \mathbf{e}_2 \\ \mathbf{e}_1 \\ \mathbf{e}_2 \end{pmatrix} = (0, 1, 1, 0, 0, 1)^\top. \end{aligned}$$

Re-encoding the resulting stacks from part (a) via Eq. (27) yields the embeddings:

	$\gamma_1 = \langle 2$	$\gamma_2 = \langle 1 \langle 1$	$\gamma_3 = \langle 2 \langle 1 \langle 2$
$\rangle_2$	$(0, 0, 0, 0, 0, 0)^\top$	reject	$(1, 0, 0, 1, 0, 0)^\top$
$\rangle_1$	reject	$(1, 0, 1, 0, 0, 0)^\top$	reject
$\langle 2$	$(0, 1, 0, 1, 0, 0)^\top$	reject	reject

Table 2: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

- d) Let $\mathbf{I}_k$ and $\mathbf{0}_k$ denote the $k \times k$ identity and zero matrices, and write $\mathbf{h}$ as m stacked k -blocks where block j is the j -th symbol from the top of the stack. Block 1 is the top, blocks beyond the stack depth are zero. Pushing onto the stack means *shifting all k -blocks one position down* and writing the new symbol into block 1. Popping means *shifting all k -blocks one position up* and zeroing block m . Realize this with two block-banded $km \times km$ matrices,

$$\mathbf{U}_{\text{push}} = \begin{pmatrix} \mathbf{0}_k & \mathbf{0}_k & \cdots & \mathbf{0}_k & \mathbf{0}_k \\ \mathbf{I}_k & \mathbf{0}_k & \cdots & \mathbf{0}_k & \mathbf{0}_k \\ \mathbf{0}_k & \mathbf{I}_k & \cdots & \mathbf{0}_k & \mathbf{0}_k \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0}_k & \mathbf{0}_k & \cdots & \mathbf{I}_k & \mathbf{0}_k \end{pmatrix}, \quad \mathbf{U}_{\text{pop}} = \begin{pmatrix} \mathbf{0}_k & \mathbf{I}_k & \mathbf{0}_k & \cdots & \mathbf{0}_k \\ \mathbf{0}_k & \mathbf{0}_k & \mathbf{I}_k & \cdots & \mathbf{0}_k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_k & \mathbf{0}_k & \mathbf{0}_k & \cdots & \mathbf{I}_k \\ \mathbf{0}_k & \mathbf{0}_k & \mathbf{0}_k & \cdots & \mathbf{0}_k \end{pmatrix},$$

and define the recurrence-matrix function $\mathbf{U} : \bar{\Sigma} \rightarrow \mathbb{R}^{km \times km}$ by

$$\mathbf{U}(y_t) = \begin{cases} \mathbf{U}_{\text{push}} & \text{if } y_t \in \{\langle 1, \dots, \langle k\}, \\ \mathbf{U}_{\text{pop}} & \text{if } y_t \in \{\rangle_1, \dots, \rangle_k\}, \\ \mathbf{I}_{km} & \text{if } y_t = \text{EOS}. \end{cases}$$

The choice $\mathbf{U}(\text{EOS}) = \mathbf{I}_{km}$ keeps the stack representation intact at termination, since EOS is not a stack operation. The input matrix $\mathbf{V} \in \mathbb{R}^{km \times (2k+1)}$ injects the new opening bracket into block 1 only when y_t is an opening bracket, so we set

$$\mathbf{V} = \begin{pmatrix} \mathbf{I}_k \\ \mathbf{0}_k \\ \vdots \\ \mathbf{0}_k \end{pmatrix} (\mathbf{I}_k \quad \mathbf{0}_k \quad \mathbf{0}_{k \times 1}) = \begin{pmatrix} \mathbf{I}_k & \mathbf{0}_k & \mathbf{0}_{k \times 1} \\ \mathbf{0}_k & \mathbf{0}_k & \mathbf{0}_{k \times 1} \\ \vdots & \vdots & \vdots \\ \mathbf{0}_k & \mathbf{0}_k & \mathbf{0}_{k \times 1} \end{pmatrix},$$

where the columns are ordered as $\langle_1, \dots, \langle_k, \rangle_1, \dots, \rangle_k, \text{EOS}$. With this $\mathbf{V}$, $\mathbf{V}\mathbf{o}_{\langle_i} = (\mathbf{e}_i, \mathbf{0}_{(m-1)k})^\top$ and $\mathbf{V}\mathbf{o}_{\rangle_i} = \mathbf{V}\mathbf{o}_{\text{EOS}} = \mathbf{0}_{km}$.

Correspondence with $\mathcal{A}$ Suppose $\mathbf{h}_{t-1}$ encodes a non-rejecting state $q = [\langle_{i_1} \dots \langle_{i_{m'}}] \in \mathcal{Q}$ via Eq. (27). We check the three transition kinds.

Valid push. If $y_t = \langle_i$ and $m' < m$, $\mathcal{A}$ moves to $q' = [\langle_{i_1} \dots \langle_{i_{m'}} \langle_i]$. Multiplying out, $\mathbf{U}_{\text{push}} \mathbf{h}_{t-1}$ takes block j of $\mathbf{h}_{t-1}$ into block $j+1$ for $j = 1, \dots, m-1$ and zeros block 1. Adding $\mathbf{V}\mathbf{o}_{\langle_i}$ writes $\mathbf{e}_i$ into block 1. The resulting blocks are $(\mathbf{e}_i, \mathbf{e}_{i_{m'}}, \dots, \mathbf{e}_{i_1}, \mathbf{0}, \dots, \mathbf{0})$, exactly $h(q')$.

Valid pop. If $y_t = \rangle_i$ with $i = i_{m'}$, $\mathcal{A}$ moves to $q' = [\langle_{i_1} \dots \langle_{i_{m'-1}}]$. Now $\mathbf{U}_{\text{pop}} \mathbf{h}_{t-1}$ takes block j into block $j-1$ for $j = 2, \dots, m$ and zeros block m , while $\mathbf{V}\mathbf{o}_{\rangle_i} = \mathbf{0}$. The resulting blocks are $(\mathbf{e}_{i_{m'-1}}, \dots, \mathbf{e}_{i_1}, \mathbf{0}, \dots, \mathbf{0})$, exactly $h(q')$.

EOS . The DFA does not consume EOS (cf. part (b)). We preserve the stack via $\mathbf{U}(\text{EOS}) = \mathbf{I}_{km}$ and $\mathbf{V}\mathbf{o}_{\text{EOS}} = \mathbf{0}$, so $\mathbf{h}_t = \mathbf{h}_{t-1}$.

Invalid transitions Pushing on a full stack ($m' = m$) loses block m (shifted out), and popping a mismatched or empty stack discards the wrong $\mathbf{e}$ or stays at $\mathbf{0}$. In either case $\mathbf{h}_t$ no longer encodes valid stack. The dynamics map alone cannot detect this. It is the recognition module of part (f) that assigns probability $< \eta$ to such continuations, so in the locally η -truncated support these strings are rejected.

Hence, on every *valid* prefix, the RNN's hidden states are in one-to-one correspondence with the states $\mathcal{A}$ visits, and the recurrence $\mathbf{h}_t = \mathbf{U}(y_t)\mathbf{h}_{t-1} + \mathbf{V}\mathbf{o}_{y_t}$ implements δ .

- e) Since $\mathbf{U}$ is now fixed, it can't depend on whether y_t should push or pop. Following the hint, double the hidden state to $\mathbb{R}^{2km}$ and perform *both* operations simultaneously in each half:

$$\mathbf{h}_t = \begin{pmatrix} \mathbf{h}_t^{(p)} \\ \mathbf{h}_t^{(q)} \end{pmatrix} \in \mathbb{R}^{2km},$$

where the top half $\mathbf{h}_t^{(p)}$ holds the result of a push and the bottom half $\mathbf{h}_t^{(q)}$ holds the result of a pop. The Heaviside activation H together with large negative entries in $\mathbf{V}$ then gates off whichever half does not match the input. We maintain the invariant that one half is non-zero and equals the correct stack encoding $h([\gamma])$ from Eq. (27). The true stack is recovered as $\mathbf{s}_t \stackrel{\text{def}}{=} \mathbf{h}_t^{(p)} + \mathbf{h}_t^{(q)}$.

Let $\mathbf{U}_{\text{push}}, \mathbf{U}_{\text{pop}} \in \mathbb{R}^{km \times km}$ be the shift matrices from part (d) and let $\mathbf{V}_{\text{ins}} \in \mathbb{R}^{km \times k}$ be the matrix that places $\mathbf{e}_i$ into block 1 when its input is $\langle_i$, i.e. $\mathbf{V}_{\text{ins}} = (\mathbf{I}_k, \mathbf{0}_k, \dots, \mathbf{0}_k)^\top$ (the first k columns of the $\mathbf{V}$ from part (d)). Define

$$\mathbf{U} = \begin{pmatrix} \mathbf{U}_{\text{push}} & \mathbf{U}_{\text{push}} \\ \mathbf{U}_{\text{pop}} & \mathbf{U}_{\text{pop}} \end{pmatrix} \in \mathbb{R}^{2km \times 2km}, \quad \mathbf{b} = -\frac{1}{2} \mathbf{1}_{2km} \in \mathbb{R}^{2km},$$

$$\mathbf{V} = \begin{pmatrix} \mathbf{V}_{\text{ins}} & -2 \mathbf{1}_{km \times k} & -2 \mathbf{1}_{km \times 1} \\ -2 \mathbf{1}_{km \times k} & \mathbf{0}_{km \times k} & -2 \mathbf{1}_{km \times 1} \end{pmatrix} \in \mathbb{R}^{2km \times (2k+1)},$$

where the $2k+1$ columns are ordered as $\langle_1, \dots, \langle_k, \rangle_1, \dots, \rangle_k, \text{EOS}$.

Why $\mathbf{U}$ is correct Multiplying out,

$$\mathbf{U}\mathbf{h}_{t-1} = \begin{pmatrix} \mathbf{U}_{\text{push}}(\mathbf{h}_{t-1}^{(p)} + \mathbf{h}_{t-1}^{(q)}) \\ \mathbf{U}_{\text{pop}}(\mathbf{h}_{t-1}^{(p)} + \mathbf{h}_{t-1}^{(q)}) \end{pmatrix} = \begin{pmatrix} \mathbf{U}_{\text{push}}\mathbf{s}_{t-1} \\ \mathbf{U}_{\text{pop}}\mathbf{s}_{t-1} \end{pmatrix},$$

so we recover both candidate next-stacks from the true stack $\mathbf{s}_{t-1}$. One of $\mathbf{h}_{t-1}^{(p)}, \mathbf{h}_{t-1}^{(q)}$ is $\mathbf{0}$ and the other is a 0/1 encoding, hence $\mathbf{s}_{t-1} \in \{0, 1\}^{km}$ and the entries of $\mathbf{U}\mathbf{h}_{t-1}$ lie in $\{0, 1\}$.

Correspondence with $\mathcal{A}$ Suppose $\mathbf{s}_{t-1} = h(q)$ for some non-rejecting state $q = [\langle i_1 \cdots \langle i_{m'} \rangle] \in \mathcal{Q}$.

Valid push ($y_t = \langle i, m' < m, q' = [\langle i_1 \cdots \langle i_{m'} \langle i \rangle]$). The one-hot $\mathbf{o}_{y_t}$ selects column i of $\mathbf{V}$:

$$\mathbf{V}\mathbf{o}_{\langle i} = \begin{pmatrix} \mathbf{V}_{\text{ins}} \mathbf{e}_i \\ -2 \mathbf{1}_{km} \end{pmatrix} = \begin{pmatrix} (\mathbf{e}_i, \mathbf{0}_{(m-1)k})^\top \\ -2 \mathbf{1}_{km} \end{pmatrix} \Rightarrow \mathbf{z}_t = \begin{pmatrix} \mathbf{U}_{\text{push}}\mathbf{s}_{t-1} + (\mathbf{e}_i, \mathbf{0})^\top - \frac{1}{2}\mathbf{1} \\ \mathbf{U}_{\text{pop}}\mathbf{s}_{t-1} - \frac{5}{2}\mathbf{1} \end{pmatrix}.$$

The top half: $\mathbf{U}_{\text{push}}\mathbf{s}_{t-1}$ has its block 1 zeroed out, so adding $(\mathbf{e}_i, \mathbf{0})^\top$ never causes a collision. Every entry is 0 or 1, and after subtracting $\frac{1}{2}$ they sit at $\pm\frac{1}{2}$. The Heaviside yields $h(q')$. The bottom half: each entry is $\leq 1 - \frac{5}{2} = -\frac{3}{2} < 0$, so H makes it $\mathbf{0}$. Thus $\mathbf{h}_t^{(p)} = h(q')$, $\mathbf{h}_t^{(q)} = \mathbf{0}$, and $\mathbf{s}_t = h(q')$.

Valid pop ($y_t = \rangle i, i = i_{m'}, q' = [\langle i_1 \cdots \langle i_{m'-1} \rangle]$). The one-hot picks column $k + i$:

$$\mathbf{V}\mathbf{o}_{\rangle i} = \begin{pmatrix} -2 \mathbf{1}_{km} \\ \mathbf{0}_{km} \end{pmatrix} \Rightarrow \mathbf{z}_t = \begin{pmatrix} \mathbf{U}_{\text{push}}\mathbf{s}_{t-1} - \frac{5}{2}\mathbf{1} \\ \mathbf{U}_{\text{pop}}\mathbf{s}_{t-1} - \frac{1}{2}\mathbf{1} \end{pmatrix}.$$

The top half is $\leq -\frac{3}{2} < 0$, so H zeros it. The bottom half has entries in $\{-\frac{1}{2}, +\frac{1}{2}\}$ and H recovers $\mathbf{U}_{\text{pop}}\mathbf{s}_{t-1} = h(q')$. Hence $\mathbf{h}_t^{(p)} = \mathbf{0}$, $\mathbf{h}_t^{(q)} = h(q')$, and $\mathbf{s}_t = h(q')$.

EOS. The last column of $\mathbf{V}$ is $-2 \mathbf{1}_{2km}$, so both halves of $\mathbf{z}_t$ are $\leq -\frac{3}{2}\mathbf{1}$ and $\mathbf{h}_t = \mathbf{0}_{2km}$. Since the recognition module of part (f) reads $\mathbf{h}_{t-1}$ to compute the probability of $y_t = \text{EOS}$, this loss of information at $t = T$ does not affect recognition.

Invalid transitions A push on a full stack ($m' = m$) shifts the bottom block of $\mathbf{s}_{t-1}$ off the top half (annihilated by $\mathbf{U}_{\text{push}}$), so $\mathbf{h}_t^{(p)}$ encodes a *different* length- m stack instead of the rejecting state of $\mathcal{A}$. Similarly, a mismatched or empty pop produces a top half that does not equal $h(q')$ for the true continuation. The dynamics of part (e) cannot detect these failures by themselves. This is the role of the output layer constructed in part (f), which assigns probability $< \eta$ to such invalid continuations and thus excludes them from the locally η -truncated support. On every valid prefix, however, the recurrence $\mathbf{h}_t = H(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{V}\mathbf{o}_{y_t} + \mathbf{b})$ tracks the transitions of $\mathcal{A}$ exactly via $\mathbf{s}_t$.

- f) Recall that one of the two halves of $\mathbf{h}_{t-1}$ is $\mathbf{0}$ and the other equals the encoding $\mathbf{s}_{t-1} = h(q)$ of the current automaton state q via Eq. (27). Thus $\mathbf{s}_{t-1}$ is composed of m blocks of size k , with the top of the stack in block $j = 1$ and the bottom in block $j = m$ (zero-padded above the stack depth). For brevity, we work with $\mathbf{s}_{t-1}$ as a stand-in for $\mathbf{h}_{t-1}$ below. The same logits are obtained by feeding either half (the all-zero half contributes nothing). Write $\mathbf{s}^{(j)} \in \mathbb{R}^k$ for the j -th block. Three indicators are linear in $\mathbf{s}_{t-1}$ and take values in $\{0, 1\}$ on every valid encoding:

$$\mathbb{I}_{\text{full}} = \mathbf{1}_k^\top \mathbf{s}^{(m)}, \quad \mathbb{I}_{\text{empty}} = 1 - \mathbf{1}_k^\top \mathbf{s}^{(1)}, \quad \mathbb{I}_{\text{top}=i} = \mathbf{s}_i^{(1)}.$$

Design $\mathbf{E}\mathbf{s}_{t-1} + \mathbf{b}'$ so that valid continuations yield a logit of 0 and invalid ones $-\alpha$. Order the rows of $\mathbf{E}$ and $\mathbf{b}'$ as $\langle 1, \dots, \langle k, \rangle 1, \dots, \rangle k, \text{EOS} \rangle$. Let $\mathbf{J}_k = \mathbf{1}_k \mathbf{1}_k^\top$ be the $k \times k$ ones matrix.

$$\mathbf{E} = \begin{pmatrix} \mathbf{0}_{k \times k} & \mathbf{0}_{k \times k} & \cdots & \mathbf{0}_{k \times k} & -\alpha \mathbf{J}_k \\ \alpha \mathbf{I}_k & \mathbf{0}_{k \times k} & \cdots & \mathbf{0}_{k \times k} & \mathbf{0}_{k \times k} \\ -\alpha \mathbf{1}_k^\top & \mathbf{0}_{1 \times k} & \cdots & \mathbf{0}_{1 \times k} & \mathbf{0}_{1 \times k} \end{pmatrix} \in \mathbb{R}^{(2k+1) \times km}, \quad \mathbf{b}' = \begin{pmatrix} \mathbf{0}_k \\ -\alpha \mathbf{1}_k \\ 0 \end{pmatrix}.$$

$$\begin{aligned} z_{\langle i} &= -\alpha \mathbf{1}_k^\top \mathbf{s}^{(m)} = -\alpha \mathbb{I}_{\text{full}}, \\ z_{\rangle i} &= \alpha \mathbf{s}_i^{(1)} - \alpha = -\alpha (1 - \mathbb{I}_{\text{top}=i}), \\ z_{\text{EOS}} &= -\alpha \mathbf{1}_k^\top \mathbf{s}^{(1)} = -\alpha (1 - \mathbb{I}_{\text{empty}}). \end{aligned}$$

These are 0 exactly when the corresponding symbol is an acceptable continuation under δ (push allowed when not full, pop allowed when its bracket matches the top, EOS allowed when the stack is empty), and $-\alpha$ otherwise.

Because the logit of *every* valid continuation is 0 and that of *every* invalid one is $-\alpha$, the partition function $Z = \sum_{y \in \Sigma} \exp(z_y)$ depends only on how many continuations are valid. There are three cases.

1. *Empty stack* ($m' = 0$): the k pushes and EOS are valid, the k pops are invalid; $Z_{\text{empty}} = (k+1) + ke^{-\alpha}$.
2. *Partially full stack* ($0 < m' < m$): the k pushes and the unique matching pop are valid; the $k-1$ mismatched pops and EOS are invalid; $Z_{\text{partial}} = (k+1) + ke^{-\alpha}$.
3. *Full stack* ($m' = m$): only the matching pop is valid; the k pushes, $k-1$ mismatched pops, and EOS are all invalid; $Z_{\text{full}} = 1 + 2ke^{-\alpha}$.

Choosing α Let $N_{\text{val}} \in \{k+1, 1\}$ denote the number of valid continuations in the current case. Valid and invalid probabilities are $1/Z$ and $e^{-\alpha}/Z$ respectively, and the locally η -truncated support condition $\mathcal{L}_\eta(p_{\text{SM}}) = D(k, m)$ requires

$$\frac{1}{Z} \geq \eta \text{ (valid)}, \quad \frac{e^{-\alpha}}{Z} < \eta \text{ (invalid)}.$$

As $\alpha \rightarrow \infty$, $e^{-\alpha} \rightarrow 0$, so $Z \rightarrow N_{\text{val}}$, the valid probability tends to $1/N_{\text{val}} \in \{\frac{1}{k+1}, 1\}$, and the invalid probability tends to 0. Since the assignment guarantees $\eta < \frac{1}{k+1} \leq 1/N_{\text{val}}$, the limit valid probability exceeds η in every case, and the limit invalid probability is below η . By continuity, there exists $\alpha^*(\eta) > 0$ s.t. both inequalities hold in all three cases simultaneously for every $\alpha \geq \alpha^*(\eta)$, so the construction goes through for sufficiently large α (cf. the footnote in the assignment). Two conditions translate into upper bounds on $e^{-\alpha}$:

Valid condition ($1/Z \geq \eta$, i.e. $Z \leq 1/\eta$):

$$e^{-\alpha} \leq \frac{1/\eta - (k+1)}{k} \quad (\text{Cases 1, 2}), \quad e^{-\alpha} \leq \frac{1/\eta - 1}{2k} \quad (\text{Case 3}).$$

Invalid condition ($e^{-\alpha}/Z < \eta$, i.e. $e^{-\alpha} < \eta Z$):

$$e^{-\alpha} < \frac{\eta(k+1)}{1-k\eta} \quad (\text{Cases 1, 2; } \eta < \frac{1}{k+1} < \frac{1}{k} \text{ ensures } 1 - k\eta > 0),$$

$$e^{-\alpha} < \frac{\eta}{1-2k\eta} \quad (\text{Case 3; automatic when } \eta \geq \frac{1}{2k}).$$

Taking α at least the maximum of the four logarithmic thresholds with strict inequality where required suffices to guarantee $\mathcal{L}_\eta(p_{\text{SM}}) = D(k, m)$.

- g) For $D(4, 3)$ we have $k = 4$ and $m = 3$, so each bracket is encoded in $\lceil \log_2 k \rceil + 1 = 3$ bits via Eq. (33) and the hidden state lives in $\mathbb{R}^{(\lceil \log_2 k \rceil + 1) \cdot m} = \mathbb{R}^9$. The binary representations:

$$\mathbf{e}_1 = (0, 0, 1)^\top, \quad \mathbf{e}_2 = (0, 1, 0)^\top, \quad \mathbf{e}_3 = (0, 1, 1)^\top, \quad \mathbf{e}_4 = (1, 0, 0)^\top.$$

Per Eq. (33), the top of the stack occupies the first 3-block of $\mathbf{h}$, with subsequent blocks descending the stack and unused slots padded with $\mathbf{0}_3$. Applying the part-(a) rules with the rightmost symbol of γ being the top, and $m = 3$ as the depth bound and re-encoding gives:

	$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_3 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_4$
$\rangle_1$	reject	$(0, 1, 1, 0, 0, 1, 0, 0, 0)^\top$	reject
$\rangle_4$	reject	reject	$(0, 0, 1, 0, 1, 0, 0, 0, 0)^\top$
$\langle_3$	$(0, 1, 1, 0, 1, 0, 0, 0, 0)^\top$	reject	reject

Table 3: The first row of the table represents different stack configurations, while the first column lists the newly read symbols. Each cell holds the binary-encoded embedding of the resulting stack via Eq. (33), or “reject” when the automaton would reject.

- h) Write $B \stackrel{\text{def}}{=} \lceil \log_2 k \rceil + 1$ for the number of bits used by Eq. (33) to encode a single bracket identity. The single-stack now lives in $\mathbb{R}^{mB}$ (B -bit block per stack) instead of $\mathbb{R}^{km}$. Following the do push and pop simultaneously as in part (e), we double the hidden state to $\mathbb{R}^{2mB}$.

$$\mathbf{U} \in \mathbb{R}^{2mB \times 2mB}, \quad \mathbf{V} \in \mathbb{R}^{2mB \times (2k+1)}, \quad \mathbf{b} \in \mathbb{R}^{2mB}.$$

The number of input columns of $\mathbf{V}$ is unchanged at $2k + 1$, since $\mathbf{o}_{y_t}$ is still the one-hot encoding of $y_t \in \bar{\Sigma}$.

Recurrence matrix $\mathbf{U}$ The block layout is identical to part (e),

$$\mathbf{U} = \begin{pmatrix} \mathbf{U}_{\text{push}} & \mathbf{U}_{\text{push}} \\ \mathbf{U}_{\text{pop}} & \mathbf{U}_{\text{pop}} \end{pmatrix},$$

and $\mathbf{U}_{\text{push}}, \mathbf{U}_{\text{pop}} \in \mathbb{R}^{mB \times mB}$ are still block-banded shift matrices. The only difference is the block size: the identity blocks $\mathbf{I}_k$ used in part (e) are replaced by $\mathbf{I}_B$, so the shift now moves B -bit segments (rather than k -bit one-hots) up or down the stack.

Input matrix $\mathbf{V}$ The block structure mirrors part (e),

$$\mathbf{V} = \begin{pmatrix} \mathbf{V}_{\text{ins}} & -2 \mathbf{1}_{mB \times k} & -2 \mathbf{1}_{mB \times 1} \\ -2 \mathbf{1}_{mB \times k} & \mathbf{0}_{mB \times k} & -2 \mathbf{1}_{mB \times 1} \end{pmatrix},$$

but two things change. First, the row counts shrink from km to mB , so the -2 gating blocks are smaller. Second, the insertion sub-matrix $\mathbf{V}_{\text{ins}} \in \mathbb{R}^{mB \times k}$ no longer just copies the one-hot input into block 1: in part (e) the new top of the stack was already $\mathbf{e}_i \in \{0, 1\}^k$, so $\mathbf{V}_{\text{ins}}$ began with $\mathbf{I}_k$. Under the binary scheme the new top is the B -bit code of bracket $\langle_i$, so the leading $B \times k$ block of $\mathbf{V}_{\text{ins}}$ becomes a small lookup table $\mathbf{E}_B$ whose i -th column is the binary representation of i (the remaining blocks of $\mathbf{V}_{\text{ins}}$ stay zero):

$$\mathbf{V}_{\text{ins}} = \begin{pmatrix} \mathbf{E}_B \\ \mathbf{0}_{B \times k} \\ \vdots \\ \mathbf{0}_{B \times k} \end{pmatrix}, \quad \mathbf{E}_B = (\mathbf{e}_1 \ \mathbf{e}_2 \ \cdots \ \mathbf{e}_k) \in \{0, 1\}^{B \times k}.$$

With this change, $\mathbf{V}\mathbf{o}_{\langle_i}$ deposits the B -bit code of $\langle_i$ into block 1 of the push half and $-2 \mathbf{1}$ into the pop half, as in part (e).

Bias vector $\mathbf{b}$ The role of $\mathbf{b}$ is unchanged. It still sets the Heaviside threshold above zero so that the post-bias entries land at $\pm \frac{1}{2}$. We keep $\mathbf{b} = -\frac{1}{2} \mathbf{1}_{2mB}$, just shorter than before.

The part-(e) correctness argument still goes through Binary codes are $\{0, 1\}$ -vectors, so $\mathbf{s}_{t-1}$ is still binary, $\mathbf{U}_{\text{push}}\mathbf{s}_{t-1}$ has block 1 equal to $\mathbf{0}_B$, and adding the binary code in block 1 cannot cause an entry to exceed 1. Hence the post-activation entries continue to lie in $\{-\frac{1}{2}, +\frac{1}{2}\}$ on the active half and at most $-\frac{3}{2}$ on the gated half, matching the part-(e).

- i) Write $B = \lceil \log_2 k \rceil + 1$ and let $\mathbf{s}_{t-1} = \mathbf{h}_{t-1}^{(p)} + \mathbf{h}_{t-1}^{(q)} \in \{0, 1\}^{mB}$ be the true binary-encoded stack from part (h). Write $\mathbf{s}^{(j)} \in \{0, 1\}^B$ for the j -th B -bit block, so $\mathbf{s}^{(1)}$ is the top of the stack and $\mathbf{s}^{(m)}$ is the bottom.

Extending the encoding (bipolar transform) Unlike the one-hot case in part (f), a single dot product $\mathbf{s}^{(1)\top} \mathbf{e}_i$ cannot decide whether two B -bit codes are equal: it counts only matching 1's. Extend each binary code to its *bipolar* form $\bar{\mathbf{e}}_i \stackrel{\text{def}}{=} 2\mathbf{e}_i - \mathbf{1}_B \in \{-1, +1\}^B$.

$$\mathbf{s}^\top \bar{\mathbf{e}}_i - \mathbf{1}_B^\top \mathbf{e}_i = 2\mathbf{s}^\top \mathbf{e}_i - \mathbf{1}_B^\top \mathbf{s} - \mathbf{1}_B^\top \mathbf{e}_i = -d_H(\mathbf{s}, \mathbf{e}_i),$$

i.e. the negative Hamming distance between $\mathbf{s}$ and $\mathbf{e}_i$. This is 0 exactly when $\mathbf{s} = \mathbf{e}_i$ and at most -1 otherwise. Note that the empty-stack code $\mathbf{0}_B$ has $d_H(\mathbf{0}, \mathbf{e}_i) = \|\mathbf{e}_i\|_1 \geq 1$ for every $i \in \{1, \dots, k\}$, since the $+1$ in B guarantees that no valid code is the all-zero vector.

Output matrix Stack the codes row-wise into $\mathbf{C} \in \{0, 1\}^{k \times B}$ with $\mathbf{C}_{i,:} = \mathbf{e}_i^\top$, and let $\mathbf{J}_{k \times B} = \mathbf{1}_k \mathbf{1}_B^\top$ be the all-ones matrix. Build the per-stack output matrix $\tilde{\mathbf{E}} \in \mathbb{R}^{(2k+1) \times mB}$ with three row blocks (pushes, pops, EOS) and m column blocks of size B :

$$\tilde{\mathbf{E}} = \begin{pmatrix} \mathbf{0}_{k \times B} & \mathbf{0}_{k \times B} & \cdots & \mathbf{0}_{k \times B} & -\alpha \mathbf{J}_{k \times B} \\ \alpha(2\mathbf{C} - \mathbf{J}_{k \times B}) & \mathbf{0}_{k \times B} & \cdots & \mathbf{0}_{k \times B} & \mathbf{0}_{k \times B} \\ -\alpha \mathbf{1}_B^\top & \mathbf{0}_{1 \times B} & \cdots & \mathbf{0}_{1 \times B} & \mathbf{0}_{1 \times B} \end{pmatrix}.$$

Because $\mathbf{h}_{t-1} = (\mathbf{h}_{t-1}^{(p)}, \mathbf{h}_{t-1}^{(q)})^\top$, horizontal duplication recovers $\mathbf{s}_{t-1}$:

$$\mathbf{E} \stackrel{\text{def}}{=} (\tilde{\mathbf{E}} \quad \tilde{\mathbf{E}}) \in \mathbb{R}^{(2k+1) \times 2mB}, \quad \mathbf{E} \mathbf{h}_{t-1} = \tilde{\mathbf{E}} \mathbf{s}_{t-1}.$$

The bias compensates the $-\mathbf{1}_B^\top \mathbf{e}_i$ term that completes the negative-Hamming-distance identity for each pop:

$$\mathbf{b}' = \begin{pmatrix} \mathbf{0}_k \\ -\alpha \mathbf{C} \mathbf{1}_B \\ 0 \end{pmatrix} \in \mathbb{R}^{2k+1}, \quad (\mathbf{C} \mathbf{1}_B)_i = \|\mathbf{e}_i\|_1.$$

Logits for valid and invalid continuations Reading off the three blocks:

$$\begin{aligned} z_{\langle i} &= -\alpha \mathbf{1}_B^\top \mathbf{s}^{(m)} = \begin{cases} 0 & \text{if } \mathbf{s}^{(m)} = \mathbf{0}_B \text{ (stack not full, push valid)} \\ -\alpha \|\mathbf{e}_j\|_1 \leq -\alpha & \text{if } \mathbf{s}^{(m)} = \mathbf{e}_j \text{ (stack full, push invalid)} \end{cases} \\ z_{\rangle i} &= \alpha (2\mathbf{e}_i^\top \mathbf{s}^{(1)} - \mathbf{1}_B^\top \mathbf{s}^{(1)}) - \alpha \|\mathbf{e}_i\|_1 = -\alpha d_H(\mathbf{s}^{(1)}, \mathbf{e}_i) \\ &= \begin{cases} 0 & \text{if } \mathbf{s}^{(1)} = \mathbf{e}_i \text{ (matching pop, valid)} \\ -\alpha d \leq -\alpha & \text{otherwise (mismatched pop or underflow, invalid)} \end{cases} \\ z_{\text{EOS}} &= -\alpha \mathbf{1}_B^\top \mathbf{s}^{(1)} = \begin{cases} 0 & \text{if } \mathbf{s}^{(1)} = \mathbf{0}_B \text{ (empty stack, EOS valid)} \\ -\alpha \|\mathbf{e}_j\|_1 \leq -\alpha & \text{if } \mathbf{s}^{(1)} = \mathbf{e}_j \text{ (non-empty, EOS invalid)} \end{cases} \end{aligned}$$

where $d \geq 1$ in every invalid case since two distinct B -bit codes, or $\mathbf{0}_B$ and any code $\mathbf{e}_j$, must differ in at least one bit. Hence *every* valid continuation has logit exactly 0 and *every* invalid continuation has logit $\leq -\alpha$, mirroring the part-(f) split. Following the same asymptotic argument as in part (f), for sufficiently large α the softmax assigns each valid continuation probability $\geq \eta$ and each invalid one probability $< \eta$, so $\mathcal{L}_\eta(p_{\text{SM}}) = D(k, m)$.

Question 4: Limitations of Transformers on Simple Languages

a) Absorb the value projection $\mathbf{W}_V^{(\ell)}$ into the MLP $\mathbf{f}$ by exhibiting a function $\mathbf{f}'$ s.t.

$$\mathbf{x}_t^{(\ell+1)} = \mathbf{f}'\left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)}, \mathbf{x}_t^{(\ell)}\right) = \mathbf{f}\left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{v}_{t'}^{(\ell)}, \mathbf{x}_t^{(\ell)}\right)$$

Express $\mathbf{v}_{t'}^{(\ell)}$ in terms of $\mathbf{x}_{t'}^{(\ell)}$ From Eq. (39c), $\mathbf{V}^{(\ell)} = \mathbf{X}^{(\ell)} \mathbf{W}_V^{(\ell)}$. Reading off the t' -th row (where the rows of $\mathbf{V}^{(\ell)}$ and $\mathbf{X}^{(\ell)}$ are $\mathbf{v}_{t'}^{(\ell)\top}$ and $\mathbf{x}_{t'}^{(\ell)\top}$ respectively), we get

$$\mathbf{v}_{t'}^{(\ell)\top} = \mathbf{x}_{t'}^{(\ell)\top} \mathbf{W}_V^{(\ell)} \iff \mathbf{v}_{t'}^{(\ell)} = \mathbf{W}_V^{(\ell)\top} \mathbf{x}_{t'}^{(\ell)}$$

Pull $\mathbf{W}_V^{(\ell)\top}$ out of the attention sum Substituting into Eq. (39e), and using that $s_{t,t'}^{(\ell)} \in \mathbb{R}$ is a scalar and matrix multiplication is linear,

$$\mathbf{z}_t^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{v}_{t'}^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{W}_V^{(\ell)\top} \mathbf{x}_{t'}^{(\ell)} = \mathbf{W}_V^{(\ell)\top} \left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)} \right)$$

Define $\mathbf{f}'$ Plugging this into Eq. (39f), $\mathbf{x}_t^{(\ell+1)} = \mathbf{f}\left(\mathbf{W}_V^{(\ell)\top} \sum_{t'} s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)}, \mathbf{x}_t^{(\ell)}\right)$ Hence

$$\mathbf{f}'(\mathbf{a}, \mathbf{b}) \stackrel{\text{def}}{=} \mathbf{f}\left(\mathbf{W}_V^{(\ell)\top} \mathbf{a}, \mathbf{b}\right) \quad \text{for } \mathbf{a}, \mathbf{b} \in \mathbb{R}^D$$

satisfies Eq. (44), absorbing the linear projection $\mathbf{W}_V^{(\ell)}$ into the first argument of the MLP.

Lipschitz constant of $\mathbf{f}'$ Since $\mathbf{f}$ is $L_{\mathbf{f}}$ -Lipschitz and the linear map $(\mathbf{a}, \mathbf{b}) \mapsto (\mathbf{W}_V^{(\ell)\top} \mathbf{a}, \mathbf{b})$ is Lipschitz with constant $\leq \sqrt{\|\mathbf{W}_V^{(\ell)}\|_2^2 + 1}$ in the product norm, the composition of Lipschitz functions is Lipschitz. Hence $\mathbf{f}'$ is Lipschitz with constant

$$L_{\mathbf{f}'} \leq L_{\mathbf{f}} \sqrt{\|\mathbf{W}_V^{(\ell)}\|_2^2 + 1},$$

a finite quantity depending only on $L_{\mathbf{f}}$ and the operator norm of the value projection. In the sequel we abuse notation and write $L_{\mathbf{f}}$ for $L_{\mathbf{f}'}$.

b) Prove $\|\mathbf{x}_t^{(\ell)}\| \leq (2L_{\mathbf{f}})^{\ell} \|\mathbf{X}^{(0)}\|_{2,\infty}$ by induction on ℓ . After part (a), take the layer update as

$$\mathbf{x}_t^{(\ell+1)} = \mathbf{f}\left(\mathbf{z}_t^{(\ell)}, \mathbf{x}_t^{(\ell)}\right), \quad \mathbf{z}_t^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)}$$

Base case ($\ell = 0$) By Definition 10, $\|\mathbf{X}^{(0)}\|_{2,\infty} = \max_{t'} \|\mathbf{x}_{t'}^{(0)}\|$, so for every t

$$\|\mathbf{x}_t^{(0)}\| \leq \|\mathbf{X}^{(0)}\|_{2,\infty} = (2L_{\mathbf{f}})^0 \|\mathbf{X}^{(0)}\|_{2,\infty}$$

Inductive hypothesis Assume that for some $\ell \geq 0$ and every $t' \in \{1, \dots, T\}$,

$$\|\mathbf{x}_{t'}^{(\ell)}\| \leq (2L_{\mathbf{f}})^{\ell} \|\mathbf{X}^{(0)}\|_{2,\infty}$$

Inductive step ($\ell \rightarrow \ell + 1$) (i) *Bound on $\|\mathbf{z}_t^{(\ell)}\|$* The weights $s_{t,t'}^{(\ell)}$ are softmax outputs, hence $s_{t,t'}^{(\ell)} \geq 0$ and $\sum_{t'=1}^T s_{t,t'}^{(\ell)} = 1$. By the triangle inequality and the inductive hypothesis,

$$\|\mathbf{z}_t^{(\ell)}\| = \left\| \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)} \right\| \leq \sum_{t'=1}^T s_{t,t'}^{(\ell)} \|\mathbf{x}_{t'}^{(\ell)}\| \leq (2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty}$$

(ii) *Lipschitz step* Using $\mathbf{f}(\mathbf{0}, \mathbf{0}) = \mathbf{0}$ and the Lipschitz property (Definition 9) of $\mathbf{f} : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}^D$ with the standard product-space norm $\|(\mathbf{a}, \mathbf{b})\| = \sqrt{\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2}$,

$$\|\mathbf{f}(\mathbf{z}, \mathbf{x})\| = \|\mathbf{f}(\mathbf{z}, \mathbf{x}) - \mathbf{f}(\mathbf{0}, \mathbf{0})\| \leq L_f \sqrt{\|\mathbf{z}\|^2 + \|\mathbf{x}\|^2} \leq L_f (\|\mathbf{z}\| + \|\mathbf{x}\|),$$

where the last inequality follows from $\|\mathbf{z}\|^2 + \|\mathbf{x}\|^2 \leq (\|\mathbf{z}\| + \|\mathbf{x}\|)^2$.

(iii) *Combine* Applying this to $\mathbf{x}_t^{(\ell+1)} = \mathbf{f}(\mathbf{z}_t^{(\ell)}, \mathbf{x}_t^{(\ell)})$ and substituting both bounds,

$$\begin{aligned} \|\mathbf{x}_t^{(\ell+1)}\| &\leq L_f (\|\mathbf{z}_t^{(\ell)}\| + \|\mathbf{x}_t^{(\ell)}\|) \\ &\leq L_f ((2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty} + (2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty}) \\ &= (2L_f)^{\ell+1} \|\mathbf{X}^{(0)}\|_{2,\infty} \end{aligned}$$

By induction, $\|\mathbf{x}_t^{(\ell)}\| \leq (2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty}$ for every $\ell \geq 0$ and every $t \in \{1, \dots, T\}$

c) Fix a layer ℓ and a query position t . The pre-softmax logit vector is $\mathbf{l} = \mathbf{K}^{(\ell)} \mathbf{q}_t^{(\ell)} \in \mathbb{R}^T$, with t' -th entry

$$l_{t'} = \mathbf{k}_{t'}^{(\ell)\top} \mathbf{q}_t^{(\ell)}$$

Rewrite the logit in terms of $\mathbf{x}^{(\ell)}$ From Eqs. (39a) and (39b), reading off rows as in part (a) gives $\mathbf{q}_t^{(\ell)} = \mathbf{W}_Q^{(\ell)\top} \mathbf{x}_t^{(\ell)}$ and $\mathbf{k}_{t'}^{(\ell)} = \mathbf{W}_K^{(\ell)\top} \mathbf{x}_{t'}^{(\ell)}$. Substituting,

$$l_{t'} = (\mathbf{W}_K^{(\ell)\top} \mathbf{x}_{t'}^{(\ell)})^\top (\mathbf{W}_Q^{(\ell)\top} \mathbf{x}_t^{(\ell)}) = \mathbf{x}_{t'}^{(\ell)\top} \mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top} \mathbf{x}_t^{(\ell)}$$

Apply Cauchy–Schwarz and the operator-norm inequality By Cauchy–Schwarz and $\|\mathbf{M}\mathbf{v}\| \leq \|\mathbf{M}\|_2 \|\mathbf{v}\|$ for the induced 2-norm,

$$|l_{t'}| \leq \|\mathbf{x}_{t'}^{(\ell)}\| \cdot \|\mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top} \mathbf{x}_t^{(\ell)}\| \leq \|\mathbf{x}_{t'}^{(\ell)}\| \cdot \|\mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top}\|_2 \cdot \|\mathbf{x}_t^{(\ell)}\|$$

Bound the representation norms uniformly by F From part (b), $\|\mathbf{x}_t^{(\ell)}\| \leq (2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty}$. With $F \geq \max(1, (2L_f)^L) \|\mathbf{X}^{(0)}\|_{2,\infty}$, we have for every $\ell \in \{0, \dots, L\}$:

- if $2L_f \geq 1$: $(2L_f)^\ell \leq (2L_f)^L$, so $\|\mathbf{x}_t^{(\ell)}\| \leq (2L_f)^L \|\mathbf{X}^{(0)}\|_{2,\infty} \leq F$;
- if $2L_f < 1$: $(2L_f)^\ell \leq 1$, so $\|\mathbf{x}_t^{(\ell)}\| \leq \|\mathbf{X}^{(0)}\|_{2,\infty} \leq F$

Hence $\|\mathbf{x}_t^{(\ell)}\|, \|\mathbf{x}_{t'}^{(\ell)}\| \leq F$, and

$$|l_{t'}| \leq F^2 \|\mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top}\|_2$$

Define L_{att} and conclude Since the parameter matrices are fixed, the quantity

$$L_{\text{att}} \stackrel{\text{def}}{=} \max_{\ell \in \{1, \dots, L\}} \|\mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top}\|_2$$

is a finite constant that depends only on the model parameters. Setting $A \stackrel{\text{def}}{=} F^2 L_{\text{att}}$, the bound $|l_{t'}| \leq A$ holds for every entry $t' \in \{1, \dots, T\}$, hence

$$\|\mathbf{l}\|_\infty = \max_{t'} |l_{t'}| \leq F^2 L_{\text{att}} = A$$

d) Let $\mathbf{l} \in \mathbb{R}^T$ with $\|\mathbf{l}\|_\infty \leq A$ and write $\mathbf{s} = \text{softmax}(\mathbf{l})$, i.e., $s_t = \exp(l_t) / \sum_{t'=1}^T \exp(l_{t'})$.

Per-coordinate bounds on $\exp(l_t)$ Since $\|\mathbf{l}\|_\infty \leq A$, every coordinate satisfies $-A \leq l_t \leq A$. Because $\exp$ is monotonically increasing,

$$\exp(-A) \leq \exp(l_t) \leq \exp(A) \quad \text{for all } t \in \{1, \dots, T\}$$

Summing the per-coordinate bounds over $t' \in \{1, \dots, T\}$ for the softmax denominator,

$$T \exp(-A) \leq \sum_{t'=1}^T \exp(l_{t'}) \leq T \exp(A)$$

Combine for s_t The fraction $s_t = \exp(l_t) / \sum_{t'} \exp(l_{t'})$ is maximized by pairing the largest numerator with the smallest denominator, and minimized by the reverse. Using the bounds,

$$s_t \leq \frac{\exp(A)}{T \exp(-A)} = \frac{\exp(2A)}{T}, \quad s_t \geq \frac{\exp(-A)}{T \exp(A)} = \frac{\exp(-2A)}{T}$$

Hence

$$\frac{\exp(-2A)}{T} \leq s_t \leq \frac{\exp(2A)}{T} \quad \text{for every } t \in \{1, \dots, T\},$$

e) Using the simplified attention from part (a) that and $\Delta_t^{(\ell)} = \|\mathbf{z}_t^{(\ell)} - \mathbf{z}'_t^{(\ell)}\|$ as in Eq. (48).

$$\mathbf{z}_t^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)}, \quad \mathbf{z}'_t^{(\ell)} = \sum_{t'=1}^T s'_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)},$$

Add-and-subtract a cross term Insert $\pm \sum_{t'} s'_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)}$ inside the difference and group:

$$\begin{aligned} \mathbf{z}_t^{(\ell)} - \mathbf{z}'_t^{(\ell)} &= \sum_{t'=1}^T s_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)} - \sum_{t'=1}^T s'_{t,t'}^{(\ell)} \mathbf{x}_{t'}^{(\ell)} \\ &= \sum_{t'=1}^T (s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}) \mathbf{x}_{t'}^{(\ell)} + \sum_{t'=1}^T s'_{t,t'}^{(\ell)} (\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}'_{t'}^{(\ell)}) \end{aligned}$$

The first sum isolates the change in attention weights with the original representations, the second isolates the change in representations weighted by the perturbed attention.

Taking norms, applying the triangle inequality on each outer sum and on each scalar-vector product, and using $s'_{t,t'}^{(\ell)} \geq 0$ (so $|s'_{t,t'}^{(\ell)}| = s'_{t,t'}^{(\ell)}$) gives Eq. (49):

$$\Delta_t^{(\ell)} \leq \underbrace{\sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|\mathbf{x}_{t'}^{(\ell)}\|}_{\text{Difficult Part}} + \underbrace{\sum_{t'=1}^T s'_{t,t'}^{(\ell)} \|\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}'_{t'}^{(\ell)}\|}_{\text{Easy Part}}$$

f) The Difficult Part (Eq. (50)) is $\sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|\mathbf{x}_{t'}^{(\ell)}\|$. We have a uniform bound on each $\|\mathbf{x}_{t'}^{(\ell)}\|$ and definition 10 gives $\|\mathbf{X}^{(\ell)}\|_{2,\infty} = \max_{t''} \|\mathbf{x}_{t''}^{(\ell)}\|$, so for every $t' \in \{1, \dots, T\}$,

$$\|\mathbf{x}_{t'}^{(\ell)}\| \leq \|\mathbf{X}^{(\ell)}\|_{2,\infty}.$$

Since $|s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \geq 0$, multiplying through preserves the inequality termwise. Summing over t' and pulling the constant $\|\mathbf{X}^{(\ell)}\|_{2,\infty}$ out of the sum yields Eq. (51):

$$\sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|\mathbf{x}_{t'}^{(\ell)}\| \leq \sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|\mathbf{X}^{(\ell)}\|_{2,\infty} = \left(\sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \right) \|\mathbf{X}^{(\ell)}\|_{2,\infty}.$$

g) First prove the general claim and then apply it to the exponential. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be twice differentiable, and let $(a_T)_{T \in \mathbb{N}}, (b_T)_{T \in \mathbb{N}}$ be bounded with $|a_T - b_T| = \mathcal{O}(1/T)$. We claim $|f(a_T) - f(b_T)| = \mathcal{O}(1/T)$.

(i) *Uniform bound on $|f'|$* Since both sequences are bounded, fix $M > 0$ with $a_T, b_T \in [-M, M]$ for every T . Twice differentiability of f implies f' is differentiable, hence continuous. By the Extreme Value Theorem, f' attains its maximum on the compact interval $[-M, M]$:

$$K \stackrel{\text{def}}{=} \max_{x \in [-M, M]} |f'(x)| < \infty$$

(ii) *Mean value theorem* For each T , the MVT gives some c_T between a_T and b_T (so $c_T \in [-M, M]$) with $f(a_T) - f(b_T) = f'(c_T)(a_T - b_T)$. Hence

$$|f(a_T) - f(b_T)| = |f'(c_T)| |a_T - b_T| \leq K |a_T - b_T|$$

(iii) *Asymptotic conclusion* Since $|a_T - b_T| = \mathcal{O}(1/T)$ and multiplication by the constant K preserves the order,

$$|f(a_T) - f(b_T)| \leq K \cdot \mathcal{O}(1/T) = \mathcal{O}(1/T)$$

Application to $f = \exp$ The function $\exp : \mathbb{R} \rightarrow \mathbb{R}$ is C^∞ , hence twice differentiable. Applying the result with $f = \exp$ to bounded $(a_T), (b_T)$ with $|a_T - b_T| = \mathcal{O}(1/T)$ gives

$$|\exp(a_T) - \exp(b_T)| = \mathcal{O}(1/T)$$

h) Let $\mathbf{s}(\mathbf{x}) = \text{softmax}(\mathbf{x})$ and write $s_t(\mathbf{x})$ for its t -th coordinate. The boundedness assumption gives some $A > 0$ with $\|\mathbf{a}_T\|_\infty, \|\mathbf{b}_T\|_\infty \leq A$ for every T .

Mean value theorem on s_t Since s_t is C^∞ on $\mathbb{R}^T$, the multivariate MVT gives a point $\mathbf{c}_T = \lambda \mathbf{a}_T + (1 - \lambda) \mathbf{b}_T$ on the segment between $\mathbf{a}_T$ and $\mathbf{b}_T$ s.t.

$$s_t(\mathbf{a}_T) - s_t(\mathbf{b}_T) = \nabla s_t(\mathbf{c}_T)^\top (\mathbf{a}_T - \mathbf{b}_T) = \sum_{k=1}^T \frac{\partial s_t}{\partial x_k}(\mathbf{c}_T) ((\mathbf{a}_T)_k - (\mathbf{b}_T)_k)$$

Convexity of $\|\cdot\|_\infty$ also gives $\|\mathbf{c}_T\|_\infty \leq A$, so by part (d),

$$s_k(\mathbf{c}_T) \leq \frac{\exp(2A)}{T} = \mathcal{O}(1/T) \quad \text{for every } k$$

Softmax Jacobian The standard formula is $\partial s_t / \partial x_k = s_t \delta_{tk} - s_t s_k$, i.e., $s_t(1 - s_t)$ on the diagonal and $-s_t s_k$ off-diagonal. Splitting the sum at $k = t$,

$$s_t(\mathbf{a}_T) - s_t(\mathbf{b}_T) = \underbrace{s_t(\mathbf{c}_T)(1 - s_t(\mathbf{c}_T))((\mathbf{a}_T)_t - (\mathbf{b}_T)_t)}_{\text{Term 1}} - \underbrace{\sum_{k \neq t} s_t(\mathbf{c}_T) s_k(\mathbf{c}_T) ((\mathbf{a}_T)_k - (\mathbf{b}_T)_k)}_{\text{Term 2}}$$

Using $0 \leq 1 - s_t(\mathbf{c}_T) \leq 1$, the part (d) bound on $s_t(\mathbf{c}_T)$, and the hypothesis on the difference,

$$|\text{Term 1}| \leq s_t(\mathbf{c}_T) |(\mathbf{a}_T)_t - (\mathbf{b}_T)_t| = \mathcal{O}(1/T) \cdot \mathcal{O}(1/T) = \mathcal{O}(1/T^2)$$

Pull $s_t(\mathbf{c}_T)$ outside the sum, bound the differences uniformly by $\max_k |(\mathbf{a}_T)_k - (\mathbf{b}_T)_k| = \mathcal{O}(1/T)$, and use $\sum_{k \neq t} s_k(\mathbf{c}_T) \leq \sum_k s_k(\mathbf{c}_T) = 1$:

$$|\text{Term 2}| \leq s_t(\mathbf{c}_T) \left(\max_k |(\mathbf{a}_T)_k - (\mathbf{b}_T)_k| \right) \sum_{k \neq t} s_k(\mathbf{c}_T) \leq \mathcal{O}(1/T) \cdot \mathcal{O}(1/T) \cdot 1 = \mathcal{O}(1/T^2)$$

By the triangle inequality the following equals to Eq. (52):

$$|\text{softmax}(\mathbf{a}_T)_t - \text{softmax}(\mathbf{b}_T)_t| \leq |\text{Term 1}| + |\text{Term 2}| = \mathcal{O}(1/T^2),$$

- i) Let's translate the hypothesis to vector norms. The assumption $\|\mathbf{w}_{T,d}^{(\ell)}\|_\infty = \mathcal{O}(1/T)$ for every $d \in [D]$ means $|(\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'^*}^{(\ell)})_d| = \mathcal{O}(1/T)$ for every $d \in [D]$ and every $t' \neq t^*$. Since D is a model constant, summing the D squared bounds and taking the square root gives:

$$\|\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'^*}^{(\ell)}\| \leq \sqrt{D} \max_d \|\mathbf{w}_{T,d}^{(\ell)}\|_\infty = \mathcal{O}(1/T) \quad \text{for every } t' \neq t^*$$

Bound the softmax differences via part (h) Throughout this paragraph we assume the query position satisfies $t \neq t^*$, so $\|\mathbf{x}_t^{(\ell)} - \mathbf{x}_{t^*}^{(\ell)}\| = \mathcal{O}(1/T)$ on the query side too. The $t = t^*$ case is handled by a trivial bound in part (j). The attention logits $\mathbf{l}_t^{(\ell)} = \mathbf{K}^{(\ell)} \mathbf{q}_t^{(\ell)}$ are bounded by part (c). Splitting the bilinear logit difference and using part (b) to bound the remaining factors by $\mathcal{O}(1)$,

$$l_{t,t'}^{(\ell)} - l_{t,t'^*}^{(\ell)} = (\mathbf{x}_t^{(\ell)} - \mathbf{x}_{t^*}^{(\ell)})^\top \mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top} \mathbf{x}_{t'}^{(\ell)} + \mathbf{x}_t^{(\ell)\top} \mathbf{W}_K^{(\ell)} \mathbf{W}_Q^{(\ell)\top} (\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'^*}^{(\ell)}),$$

both differences are $\mathcal{O}(1/T)$ when $t, t' \neq t^*$, hence $|l_{t,t'}^{(\ell)} - l_{t,t'^*}^{(\ell)}| = \mathcal{O}(1/T)$ for $t' \neq t^*$. Applying part (h) to the resulting softmax outputs yields

$$|s_{t,t'}^{(\ell)} - s_{t,t'^*}^{(\ell)}| = \mathcal{O}(1/T^2) \quad \text{for every } t' \neq t^*$$

Bound the representations via part (b) Part (b) gives $\|\mathbf{x}_{t'}^{(\ell)}\| \leq (2L_f)^\ell \|\mathbf{X}^{(0)}\|_{2,\infty}$. Since L is fixed, this is $\mathcal{O}(1)$ uniformly in T and t' .

Substituting the two bounds into the truncated Difficult Part and using that the sum has exactly $T - 1$ terms, we get Eq. (53):

$$\sum_{\substack{t'=1 \\ t' \neq t^*}}^T |s_{t,t'}^{(\ell)} - s_{t,t'^*}^{(\ell)}| \|\mathbf{x}_{t'}^{(\ell)}\| \leq (T - 1) \cdot \mathcal{O}(1/T^2) \cdot \mathcal{O}(1) = \mathcal{O}(1/T),$$

- j) Prove Eqs. (55) and (56) jointly by induction on ℓ , then apply the bound to $\mathbf{h}(\mathbf{y})$.

Base case ($\ell = 0$) By Eq. (38), $\mathbf{x}_t^{(0)} = (\mathbf{r}(y_t), \mathbf{u}(t))$. Since $\mathbf{y}$ and $\mathbf{y}'$ differ only at t^* :

- if $t \neq t^*$: $\mathbf{x}_t^{(0)} = \mathbf{x}_{t^*}^{(0)}$, so $\|\mathbf{x}_t^{(0)} - \mathbf{x}_{t^*}^{(0)}\| = 0 = \mathcal{O}(1/T)$
- if $t = t^*$: $\|\mathbf{x}_{t^*}^{(0)} - \mathbf{x}_{t^*}^{(0)}\| = \delta = \mathcal{O}(1)$ (a constant in T , since $\mathbf{r}$ ranges over finitely many bounded embeddings).

So Eq. (56) holds at $\ell = 0$.

Inductive hypothesis Assume Eq. (56) holds at layer ℓ :

$$\|\mathbf{x}_t^{(\ell)} - \mathbf{x}_{t^*}^{(\ell)}\| = \begin{cases} \mathcal{O}(1) & \text{if } t = t^*, \\ \mathcal{O}(1/T) & \text{otherwise.} \end{cases}$$

Inductive step: bounding $\Delta_t^{(\ell)}$, Eq. (55) The bound depends on whether the query position t is the flipped one, as the bilinear logit propagation in part (i) requires the query side to also differ by $\mathcal{O}(1/T)$. Split into two cases.

Case $t \neq t^$.* Take Eq. (54b)'s four terms and bound each:

- *Term 1* ($\sum_{t' \neq t^*} |s - s'| \|\mathbf{x}_{t'}^{(\ell)}\|$): the inductive hypothesis says $\|\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'^*}^{(\ell)}\| = \mathcal{O}(1/T)$ for $t' \neq t^*$ and for the query t itself (since $t \neq t^*$), so part (i) applies and gives $\mathcal{O}(1/T)$.

- *Term 2* ($|s_{t,t^*}^{(\ell)} - s_{t,t^*}'^{(\ell)}| \|\mathbf{x}_{t^*}^{(\ell)}\|$): part (d) gives $|s_{t,t^*}^{(\ell)}|, |s_{t,t^*}'^{(\ell)}| \leq \exp(2A)/T$, so the difference is $\mathcal{O}(1/T)$, and part (b) gives $\|\mathbf{x}_{t^*}^{(\ell)}\| = \mathcal{O}(1)$. Product: $\mathcal{O}(1/T)$.
- *Term 3* ($\sum_{t' \neq t^*} s_{t,t'}^{(\ell)} \|\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'}'^{(\ell)}\|$): the inductive hypothesis bounds each $\|\mathbf{x}_{t'}^{(\ell)} - \mathbf{x}_{t'}'^{(\ell)}\| = \mathcal{O}(1/T)$, so we can pull it out and use $\sum_{t' \neq t^*} s_{t,t'}'^{(\ell)} \leq 1$. Product: $\mathcal{O}(1/T)$.
- *Term 4* ($s_{t,t^*}'^{(\ell)} \|\mathbf{x}_{t^*}^{(\ell)} - \mathbf{x}_{t^*}'^{(\ell)}\|$): part (d) gives $s_{t,t^*}'^{(\ell)} = \mathcal{O}(1/T)$ and the inductive hypothesis gives $\|\mathbf{x}_{t^*}^{(\ell)} - \mathbf{x}_{t^*}'^{(\ell)}\| = \mathcal{O}(1)$. Product: $\mathcal{O}(1/T)$.

Summing the four $\mathcal{O}(1/T)$ contributions yields $\Delta_t^{(\ell)} = \mathcal{O}(1/T)$ for $t \neq t^*$.

Case $t = t^$.* The query side $\mathbf{x}_{t^*}^{(\ell)} - \mathbf{x}_{t^*}'^{(\ell)}$ is only $\mathcal{O}(1)$ by the inductive hypothesis, so the bilinear logit propagation in part (i) does *not* apply at the query position and Term 1 cannot be sharpened. Using the trivial bound $\sum_{t' \neq t^*} |s_{t^*,t'}^{(\ell)} - s_{t^*,t'}'^{(\ell)}| \leq \sum_{t' \neq t^*} (s_{t^*,t'}^{(\ell)} + s_{t^*,t'}'^{(\ell)}) \leq 2$ together with $\|\mathbf{x}_{t'}^{(\ell)}\| \leq F = \mathcal{O}(1)$ from part (b) gives Term 1 = $\mathcal{O}(1)$. Terms 2-4 are still $\mathcal{O}(1/T)$ by the same arguments above as none of them invokes part (h) on the query side. Hence $\Delta_{t^*}^{(\ell)} = \mathcal{O}(1)$.

Combining the two cases gives Eq. (55) in the form

$$\Delta_t^{(\ell)} = \begin{cases} \mathcal{O}(1) & \text{if } t = t^*, \\ \mathcal{O}(1/T) & \text{otherwise.} \end{cases}$$

Inductive step: bounding $\|\mathbf{x}_t^{(\ell+1)} - \mathbf{x}_t'^{(\ell+1)}\|$ (Eq. (56)) By the Lipschitz step from part (b) applied to differences (rather than to $\mathbf{f}(\mathbf{0}, \mathbf{0})$),

$$\|\mathbf{x}_t^{(\ell+1)} - \mathbf{x}_t'^{(\ell+1)}\| = \|\mathbf{f}(\mathbf{z}_t^{(\ell)}, \mathbf{x}_t^{(\ell)}) - \mathbf{f}(\mathbf{z}_t'^{(\ell)}, \mathbf{x}_t'^{(\ell)})\| \leq L_f(\Delta_t^{(\ell)} + \|\mathbf{x}_t^{(\ell)} - \mathbf{x}_t'^{(\ell)}\|)$$

Splitting on the value of t and combining the two-case bound on $\Delta_t^{(\ell)}$ with the inductive hypothesis,

$$\|\mathbf{x}_t^{(\ell+1)} - \mathbf{x}_t'^{(\ell+1)}\| \leq \begin{cases} L_f(\mathcal{O}(1) + \mathcal{O}(1)) = \mathcal{O}(1) & \text{if } t = t^*, \\ L_f(\mathcal{O}(1/T) + \mathcal{O}(1/T)) = \mathcal{O}(1/T) & \text{otherwise,} \end{cases}$$

closing the induction and giving Eq. (56) at $\ell + 1$.

Implication for $\mathbf{h}(\mathbf{y})$ By definition, $\mathbf{h}(\mathbf{y}) = \mathbf{x}_T^{(L)}$. The hypothesis $t^* < T$ means $T \neq t^*$, so the otherwise case of Eq. (56) applies:

$$\|\mathbf{h}(\mathbf{y}) - \mathbf{h}(\mathbf{y}')\| = \|\mathbf{x}_T^{(L)} - \mathbf{x}_T'^{(L)}\| = \mathcal{O}(1/T)$$

The final EOS representations of a string and any one-bit-flipped variant become indistinguishable at rate $1/T$ as the sequence length grows, this is Lemma 1.

k) Apply Lemma 1 to the **Odd-1** acceptor and draw out its consequences.

(i) Parity of $\mathbf{y}'$ Suppose $\mathbf{y} \in \text{Odd-1}$, so $\mathbf{y}$ contains $2k + 1$ ones for some $k \in \mathbb{N}$. Flipping a single bit at position t^* either turns a 0 into a 1 (giving $2k + 2$ ones) or a 1 into a 0 (giving $2k$ ones). Either way, the count's parity flips from odd to even, so

$$\mathbf{y}' \notin \text{Odd-1}$$

The correct labels of $\mathbf{y}$ and $\mathbf{y}'$ are therefore opposite.

(ii) Implication for $\hat{p}'$ From part (j), $\|\mathbf{h}(\mathbf{y}) - \mathbf{h}(\mathbf{y}')\| = \mathcal{O}(1/T)$. Since the readout vector $\mathbf{w}$ is fixed (independent of T), Cauchy–Schwarz gives

$$|\mathbf{h}(\mathbf{y})^\top \mathbf{w} - \mathbf{h}(\mathbf{y}')^\top \mathbf{w}| \leq \|\mathbf{h}(\mathbf{y}) - \mathbf{h}(\mathbf{y}')\| \cdot \|\mathbf{w}\| = \mathcal{O}(1/T)$$

The two scalar logits $\mathbf{h}(\mathbf{y})^\top \mathbf{w}$ and $\mathbf{h}(\mathbf{y}')^\top \mathbf{w}$ are bounded sequences in T (since $\|\mathbf{h}\|$ is bounded by part (b) and $\|\mathbf{w}\|$ is fixed), and they differ by $\mathcal{O}(1/T)$. The sigmoid σ is C^∞ on $\mathbb{R}$, so part (g) applies and

$$|\hat{p} - \hat{p}'| = |\sigma(\mathbf{h}(\mathbf{y})^\top \mathbf{w}) - \sigma(\mathbf{h}(\mathbf{y}')^\top \mathbf{w})| = \mathcal{O}(1/T)$$

Concretely: if the model is correct on $\mathbf{y}$, i.e., $\hat{p} > 0.5$, then for sufficiently large T the gap $|\hat{p} - \hat{p}'|$ is below the margin $\hat{p} - 0.5$, forcing $\hat{p}' > 0.5$ as well. By Eqs. (41a) and (41b), the model then accepts $\mathbf{y}'$ too, but $\mathbf{y}' \notin \text{Odd-1}$, so the model misclassifies $\mathbf{y}'$.

(iii) Limit $|\mathbf{y}| \rightarrow \infty$ Taking $T = |\mathbf{y}| \rightarrow \infty$ in the bound from (ii),

$$\lim_{T \rightarrow \infty} |\hat{p} - \hat{p}'| = 0$$

The transformer assigns asymptotically same probabilities to $\mathbf{y}$ and to its one-bit-flipped variant $\mathbf{y}'$ that has opposite parity. Hence soft-attention transformers cannot recognize Odd-1 for long inputs and any decision boundary is eventually breached by a single-bit perturbation.

- 1) How layer normalization affects the chain of bounds, and what the asymptotic nature of the result actually rules out.

(i) Adding layer normalization Layer normalization, applied per token, has the form

$$\text{LN}(\mathbf{x}) = \frac{\mathbf{x} - \mu(\mathbf{x}) \mathbf{1}}{\sqrt{\sigma^2(\mathbf{x}) + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta},$$

with μ, σ^2 the empirical mean and variance of the entries of $\mathbf{x}$, and learnable $\boldsymbol{\gamma}, \boldsymbol{\beta}$. The key question is whether LN is Lipschitz continuous, since the entire proof in parts (b)–(j) depended on a finite Lipschitz constant for $\mathbf{f}$.

Case $\epsilon > 0$. The denominator $\sqrt{\sigma^2(\mathbf{x}) + \epsilon} \geq \sqrt{\epsilon} > 0$ is bounded away from zero, so LN is smooth with a bounded gradient on $\mathbb{R}^D$, hence Lipschitz with some constant L_{LN} . The composition $\mathbf{f} \circ \text{LN}$ or $\text{LN} \circ \mathbf{f}$ are then Lipschitz with constant $L_{\mathbf{f}} \cdot L_{\text{LN}}$. Replacing $L_{\mathbf{f}}$ by this product throughout every bound from (b) onward goes through unchanged and the asymptotic $\mathcal{O}(1/T)$ conclusion of Lemma 1 is preserved.

Case $\epsilon = 0$. The denominator $\sigma(\mathbf{x})$ vanishes whenever the entries of $\mathbf{x}$ are equal constant inputs, and the gradient of LN blows up near that set. So LN is *not* globally Lipschitz, and there is no finite L_{LN} to absorb. The Lipschitz step in part (j) breaks. A perturbation of size $\mathcal{O}(1/T)$ to the pre-normalization activations can be amplified arbitrarily after LN, so we cannot conclude $\Delta_t^{(\ell+1)} = \mathcal{O}(1/T)$. The $\mathcal{O}(1/T)$ guarantee is lost.

(ii) Implication for fixed-length classification The bound $|\hat{p} - \hat{p}'| = \mathcal{O}(1/T)$ is asymptotic as the gap shrinks as $T \rightarrow \infty$, but for any fixed length T^* it is merely *small*, not zero. $\|\mathbf{h}(\mathbf{y}) - \mathbf{h}(\mathbf{y}')\| \geq c/T^*$ for some constant $c > 0$, so the representations of $\mathbf{y}$ and $\mathbf{y}'$ remain distinct for every $|\mathbf{y}| \leq T^*$.

Hence a soft-attention transformer can in principle classify all strings of length up to T^* correctly by choosing the readout $\mathbf{w}$ with $\|\mathbf{w}\|$ scaling like T^* so that the small representation gap is amplified through the sigmoid past the 0.5 decision boundary. The lemma does *not* say transformers fail at parity for short strings.

What it *does* say is that no fixed parameter setting recognizes **Odd-1** for *all* lengths simultaneously. As $|\mathbf{y}|$ grows beyond T^* , the same readout no longer separates $\mathbf{y}$ from $\mathbf{y}'$, and the classifier eventually agrees on inputs of opposite parity. Recognition of **Odd-1** is therefore length-uniformly impossible, even though it is achievable on any bounded-length subset.